OBJECT DETECTION

LEARNING OBJECTIVES

→ Identify the components used for object detection (landmark, anchor, bounding box, grid...) and their purpose
→ Non-max supression to increase accuracy.
→ Intersection over union
→ Handle bounding boxes, a type of image annotation popular in deep learning
→ Apply sparse categorical crossentropy for pixelwise prediction
→ Explain the difference between a regular CNN and a U-net

deeplearning.ai

Object Detection

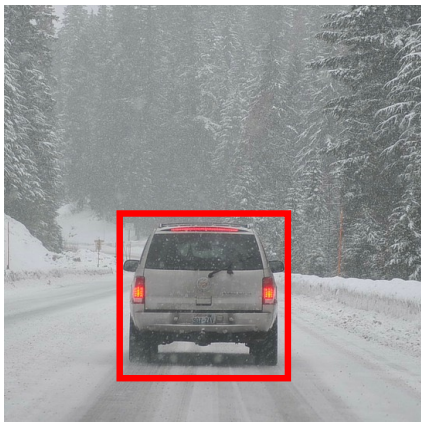
Object localization

What are localization and detection?

Image classification

Label: "car"

Classification with localization



"car" + red bounding box

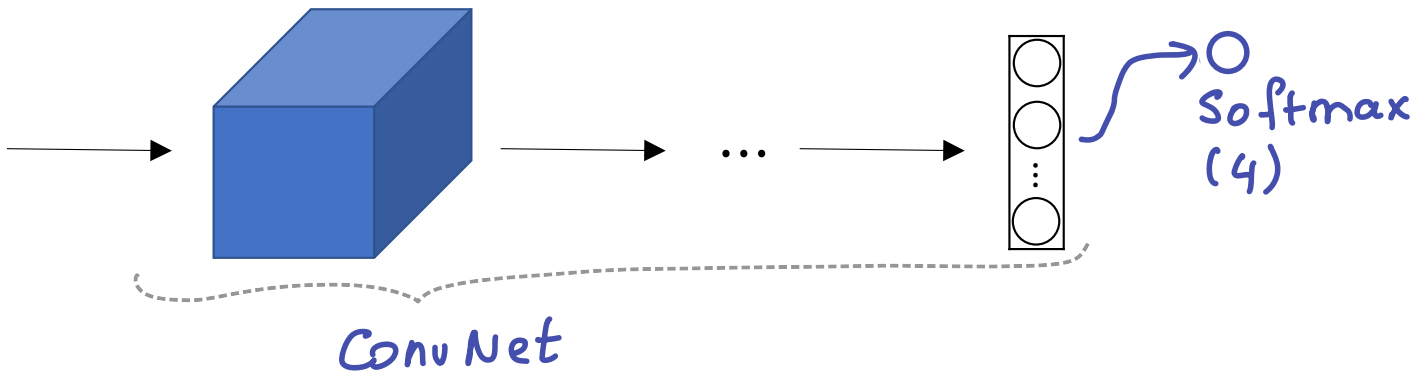
1 object

Detection

- Multiple objects
- Detect them all
- Localize them all
- May have multiple objects of different categories.

Andrew Ng

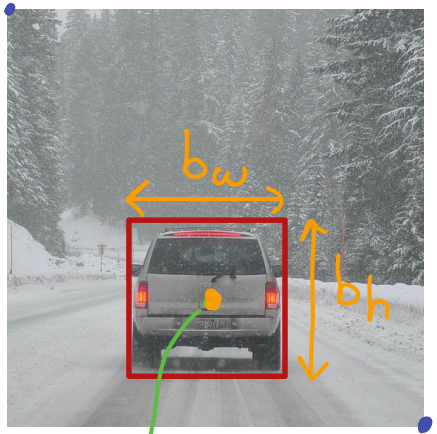
Classification with localization

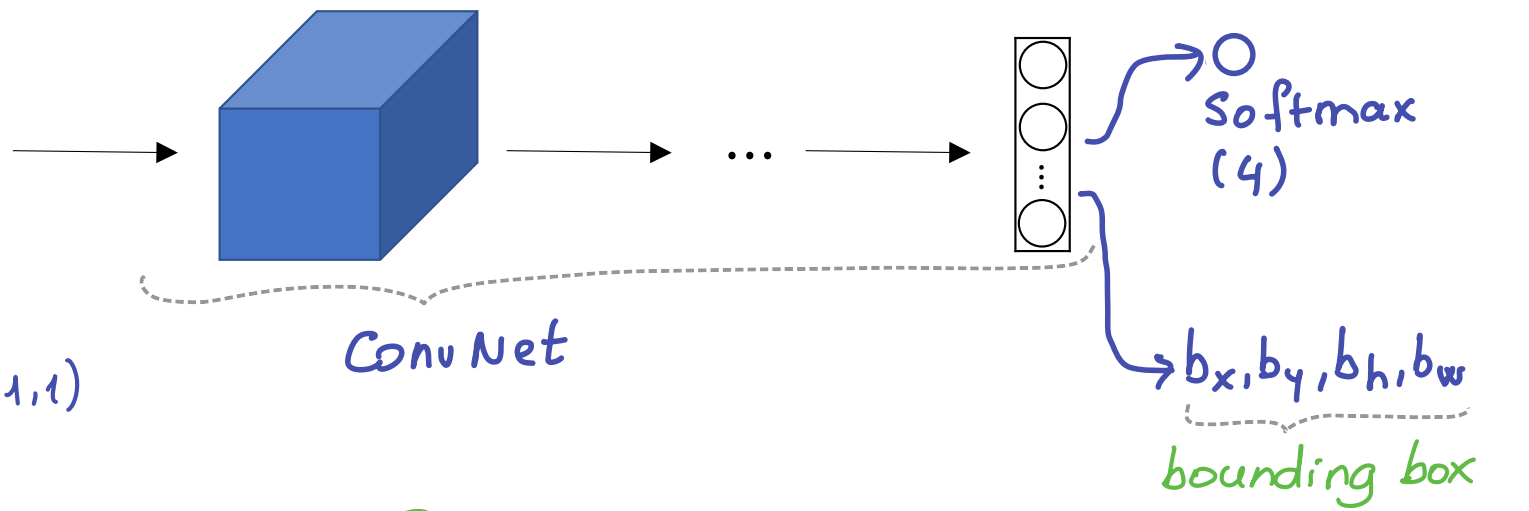


1 - pedestrian
2 - car
3 - motorcycle
4 - background (none)

Andrew Ng

Classification with localization





bx ≅ 0.5
by ≅ 0.7
bh ≅ 0.3
bw ≅ 0.4

1 - pedestrian
2 - car
3 - motorcycle
4 - background (none)

} 4 classes

- For localization change the CNN to output 4 more numbers.
- Bounding box: (b_x, b_y): center of red rectangle
  (b_h, b_w): height and width of red rectangle

Andrew Ng

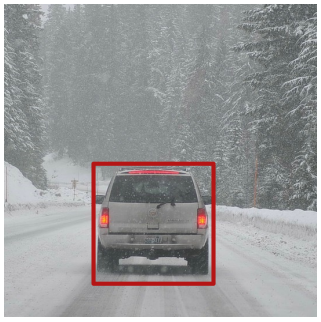
Classification with Localization (1 object)

Defining the target label y

1 - pedestrian
2 - car
3 - motorcycle
4 - background

Need to output b_x, b_y, b_h, b_w, class label (1-4)

x:



No object

Target Label: $y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$ p_c → is there any object? c_1, c_2, c_3 → classes

$y = \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$

(x, y)

$y = \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$ 0 → No object; ? → don't care

Andrew Ng

- Define the loss function

$$\mathcal{L}(\hat{y}, y) = \begin{cases} (\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 + \cdots + (\hat{y}_8 - y_8)^2, & \text{if } y_1 = 1 \quad \text{(there's an object in the image)} \\ (\hat{y}_1 - y_1)^2, & \text{if } y_1 = 0 \quad \text{(no object)} \end{cases}$$

  - if $y_1 = 0$ (no object), the other elements of y are don't care. So, all you care about is how accurately is your neural network estimating y_1, which is p_c.

  - In practice, you could use log-like loss for the c_1, c_2, c_3 of the softmax output, squared loss for bounding box coordinates and for p_c you could use a logistic regression loss.

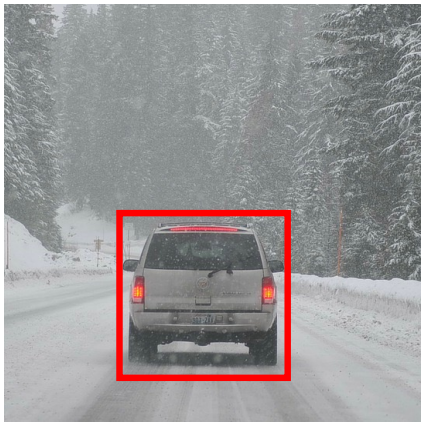
deeplearning.ai

Object Detection

Landmark detection

Goal: The NN should output x and y coordinates of important points in an image, which are called as "landmarks".

Landmark detection

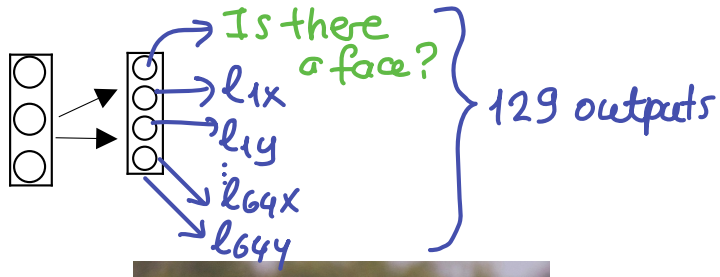


b_x, b_y, b_h, b_w

Face Recognition

positions of 64 landmarks (red pts.) on the face

l_{1x}, l_{1y}
l_{2x}, l_{2y}
l_{3x}, l_{3y}
l_{4x}, l_{4y}
$\vdots$
l_{64x}, l_{64y}



- People pose detection
- Key point positions: shoulders, chest, elbows, wrists

outputs
l_{1x}, l_{1y} (left shoulder)
l_{2x}, l_{2y} (right shoulder)
$\vdots$
l_{32x}, l_{32y}

Andrew Ng

- Landmark detection on the face is a basic building block for recognizing emotions from faces, augmented reality applications (e.g. placing a crown on the head), computer graphics effects (warping the face etc.)
- we need a labeled dataset for training, where the 64 landmarks have been laborously hand annotated.
- The labels have to be consistent across images.

- People pose detection: The red points correspond to key positions: midpoint of the chest, the left shoulder, left elbow, the wrist, and so on.

deeplearning.ai

Object Detection

Object detection

- Goal: object detection using sliding windows detection algorithm.

Car detection example

closely cropped images of a car

Training set:

x	y
	1
	1
	1
	0
	0

→ ConvNet → y

Andrew Ng

Sliding windows detection

- Feed the image block shown with the red square to the trained ConvNet for object detection.

Sliding windows detection

ConvNet → 0

- Slide the red square to the right.
- Run the ConvNet with the new image block.
- Shift the red square by using a certain stride.

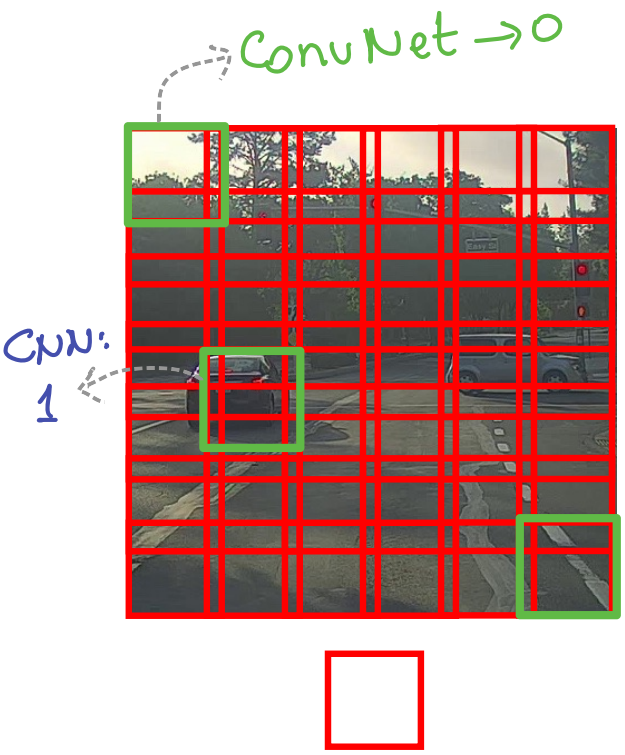
Andrew Ng

Sliding windows detection

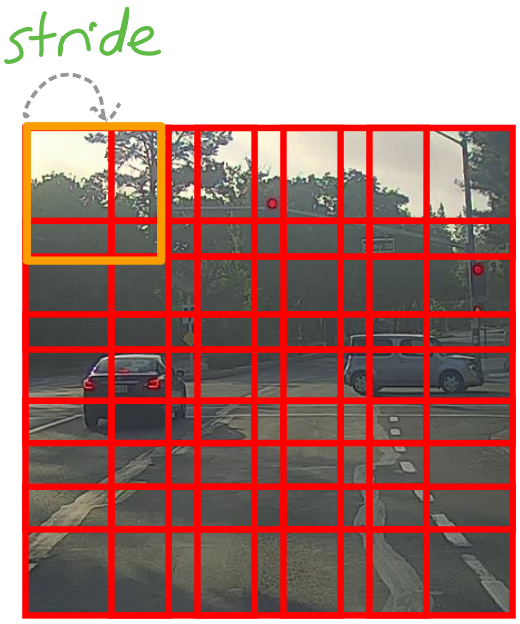
→ ConvNet → 0

• Last position of the red square.

Sliding windows detection



- Feed the regions of the image in the red squares to the ConvNet to have it classified as 0 or 1 for each position and stride.



- Use a larger window
- As long as there's a car in the image, there will be a window for which ConvNet will output 1. (we hope)

- Use a larger window

Andrew Ng

- Disadvantage of sliding windows detection: computational cost.
  - We are cropping out so many different square regions in the image and running each of them independently through a ConvNet.
  - If you use a big stride (step size) that will reduce the number of windows you need to pass through the ConvNet, but the coarser granularity may hurt the performance.
  - If you use a small stride, the computational cost will be high.

- Before deep learning, linear classifiers were used with hand-engineered features for object detection. Linear classifiers were cheap to run, so sliding windows approach was okay.
- But, using a ConvNet for classification is much more expensive and sliding windows is infeasibly slow.
- Also, unless you use a very fine granularity or very small stride, the object is not localized very well within the image.

Solution:

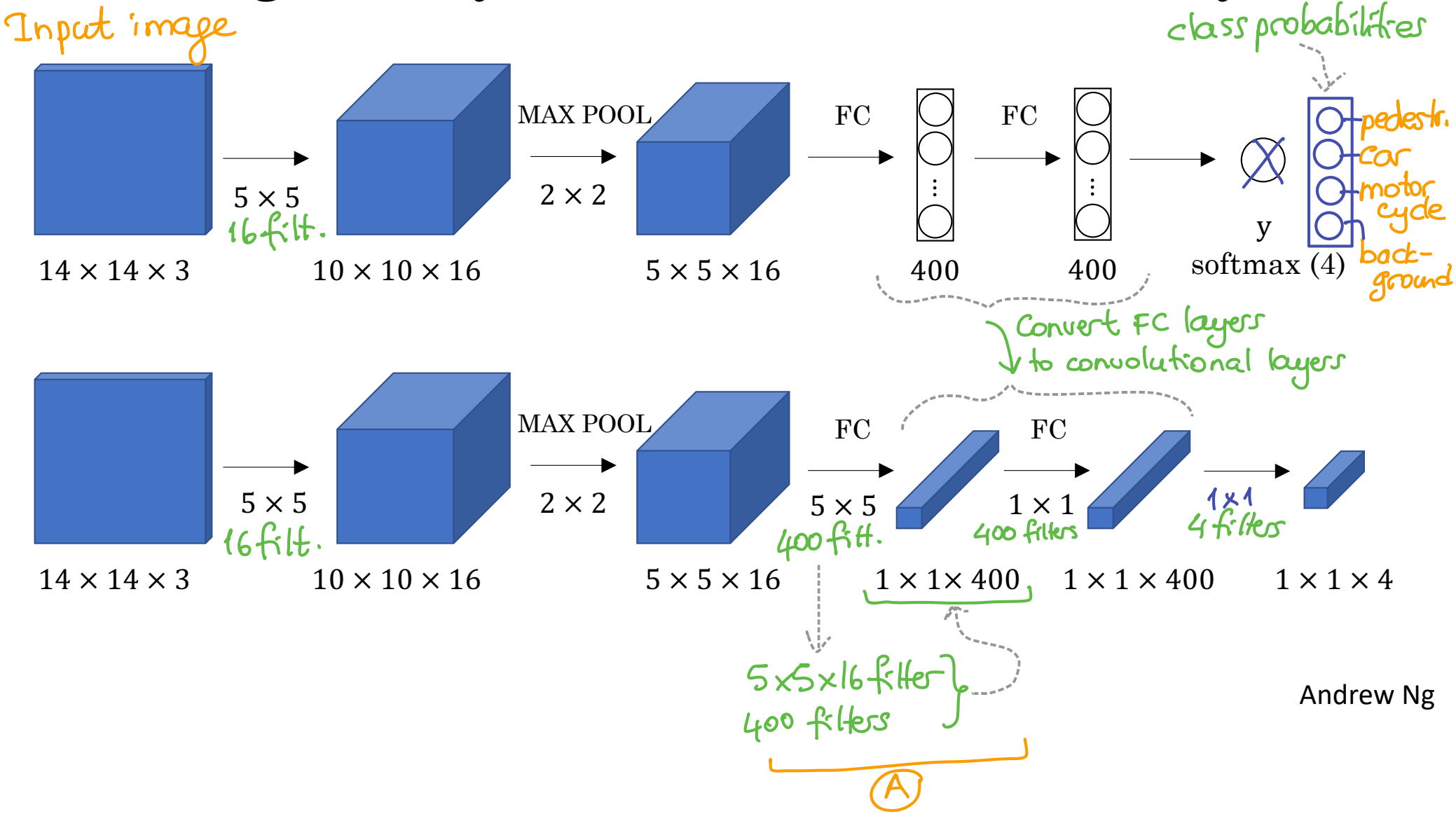
- The problem of computational cost has a pretty good solution: sliding windows can be implemented convolutionally and much more efficiently.

deeplearning.ai

Object Detection

Convolutional implementation of sliding windows

Turning FC layer into convolutional layers



Andrew Ng

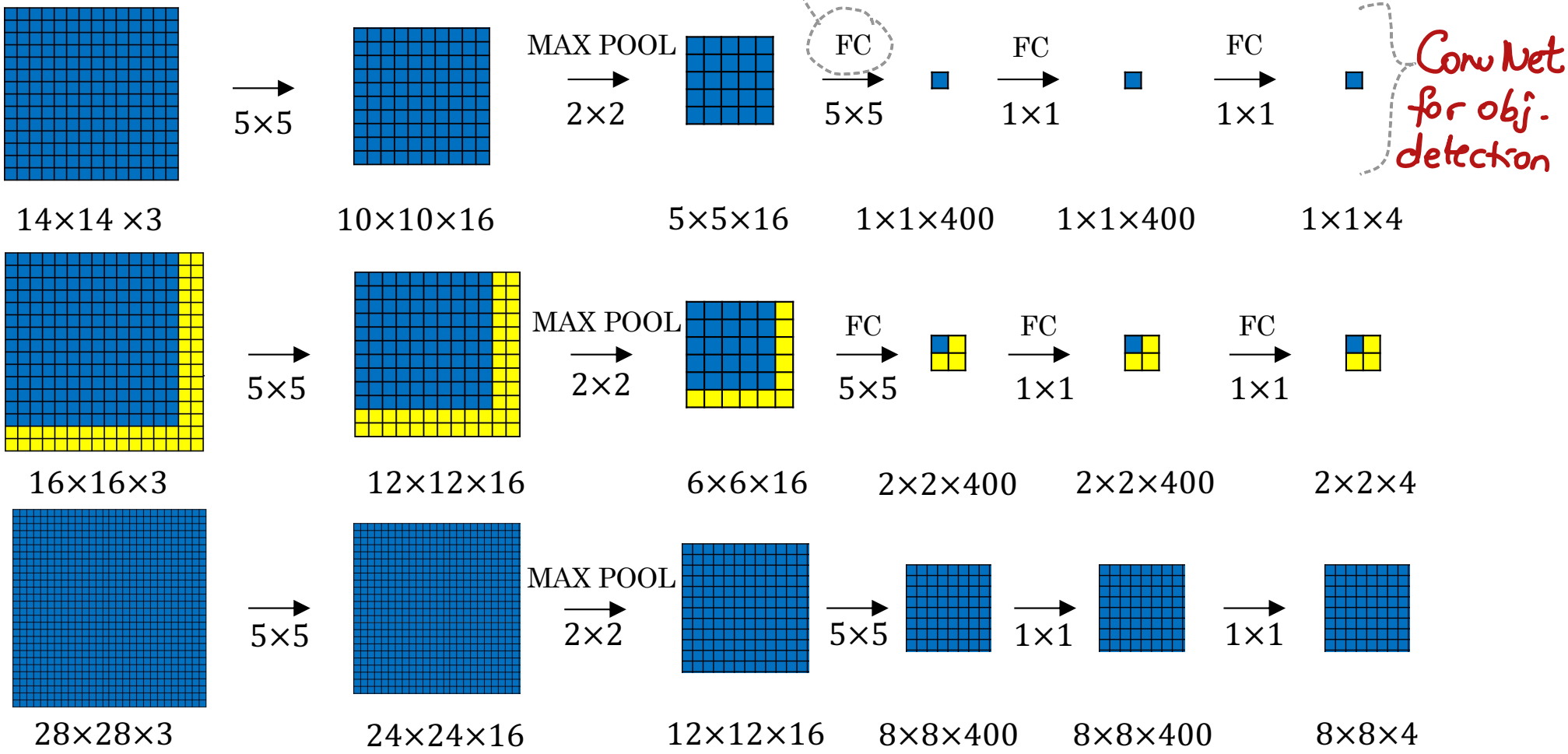
(A) To convert the first FC layer (in the top network) to convolutional layers, we used 400, 5x5x16 filters. The output is a 1x1x400 volume.

So, rather than viewing there as 400 nodes (in the above CNN), we're going to view this as a 1x1x400 volume (in the below CNN).

Mathematically, this is the same as the FC layer because each of the 400 nodes (below figure) has a filter of dimension 5x5x16. So each of these 400 values is some arbitrary linear function of these 5x5x16 activations from the previous layer.

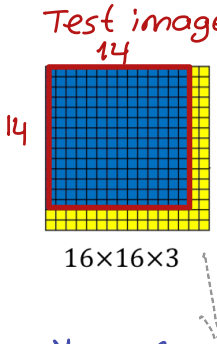
An FC layer which is turned into convolutional layers.

Convolution implementation of sliding windows

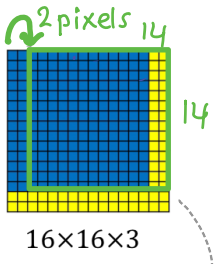


[Sermanet et al., 2014, OverFeat: Integrated recognition, localization and detection using convolutional networks]

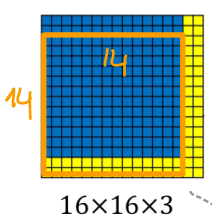
- Only the front faces of volumes are shown above.



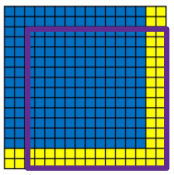
- Your ConvNet inputs 14x14x3 images and your test set image is 16x16x3 The yellow stripe is added to the border of the image.
- First we might want to put the blue region through the ConvNet and generate a classification 0 or 1.



- Slide the region to the right by 2 pixels, and input the green square into the ConvNet and get another label: 0 or 1.



- Input the orange region into the ConvNet and run it one more time to get another label: 0 or 1.

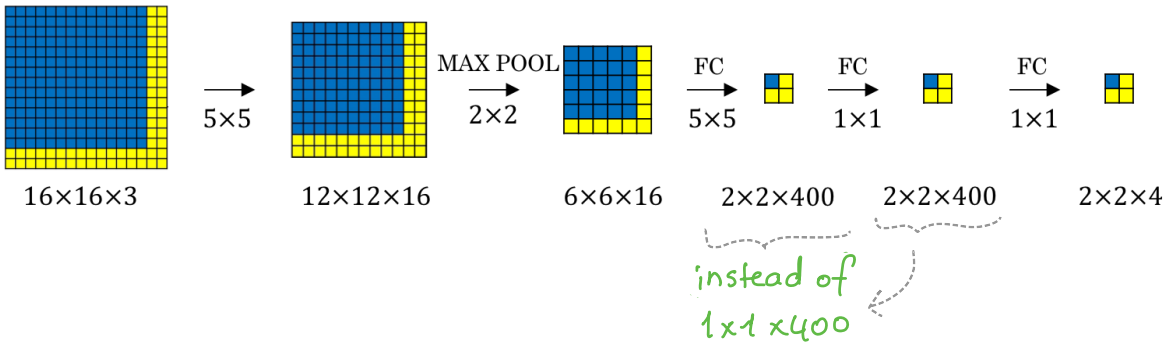


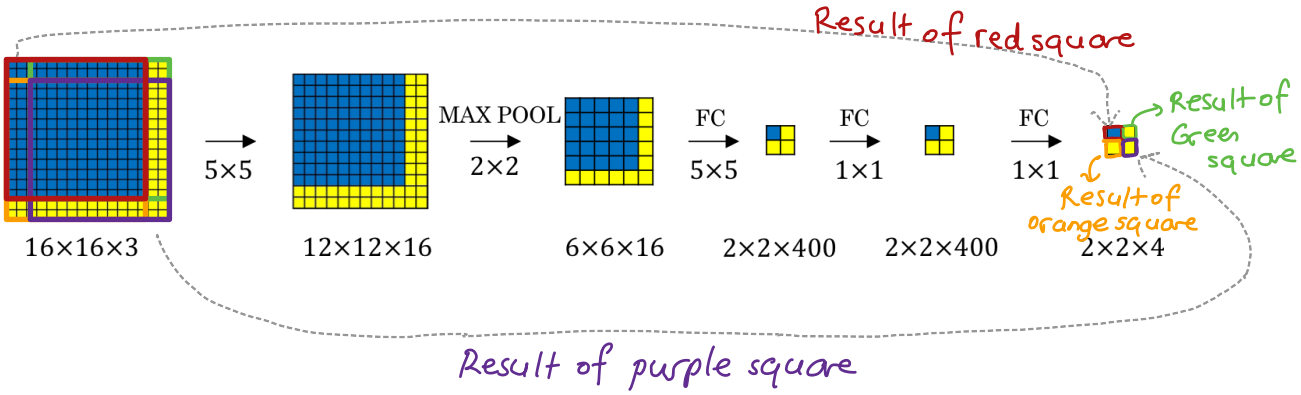
16×16×3

- Put the purple square through the ConvNet and get a classification label 0 or 1.

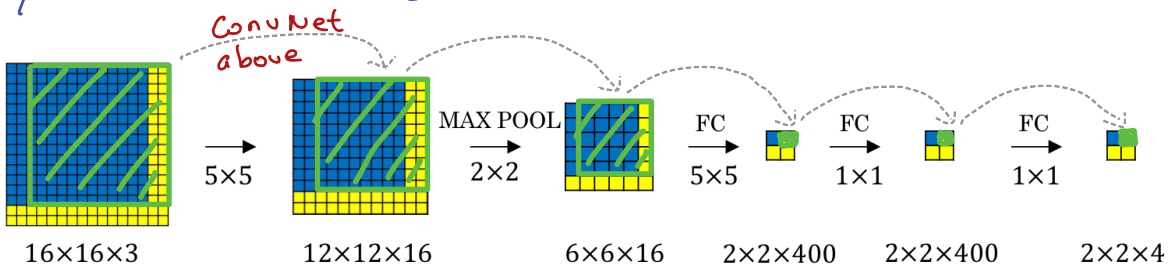
- Overall, we have run the ConvNet 4 times in order to get 4 labels.
- But the computations done by these 4 computations are highly duplicative since the 4 input square regions (red, green, orange and purple) are overlapping a lot.

- The convolution implementation of sliding windows allows to share a lot of computation among the 4 passes of ConvNet:
- Pass the extended image through the ConvNet with the same parameters and filters:



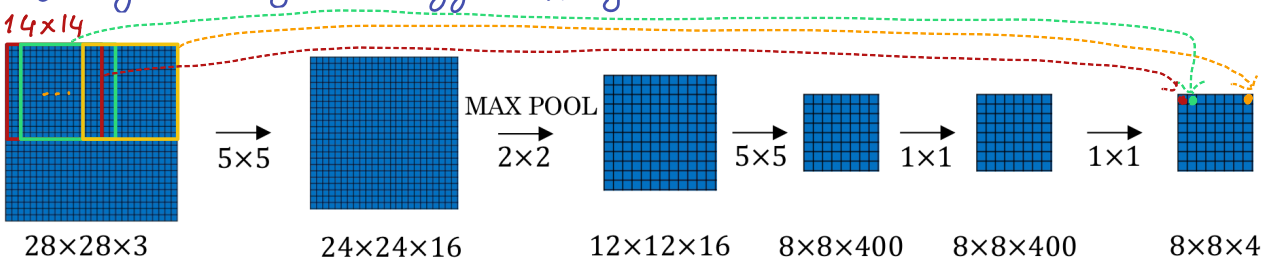


- The final 2×2×4 volume contains the classification results of the 4, 14×14×3 regions in the corresponding locations indicated by the same colors above.



- If you pass the green shaded region through the ConvNet above, the first layers activation would be the green shaded region in the second square. Then, the green squares will be computed at each layer.
- Summary: What the convolutional implementation does is, instead of forcing you to run forward propagation on 4 subsets of the image independently, it combines all 4 into one form of computation and shares a lot of the computation in the regions of the image that are common.

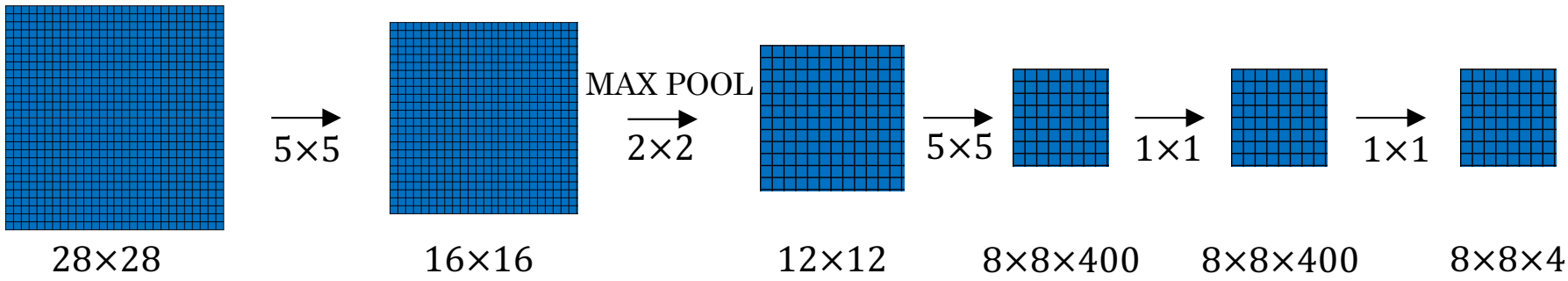
- Let's go through a bigger image: (stride = 2)

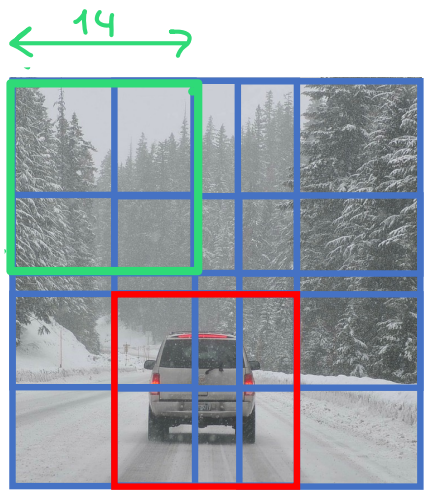


[Sermanet et al., 2014, OverFeat: Integrated recognition, localization and detection using convolutional networks]

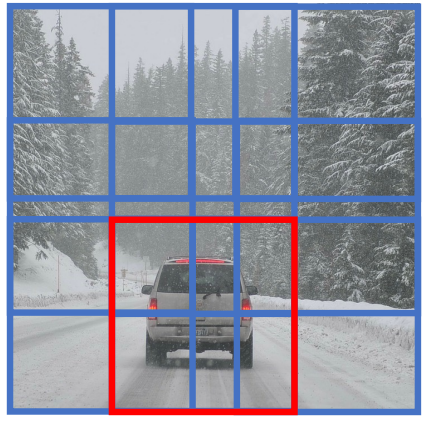
- The output will be 8×8×4: The first 14×14×3 region in the input (shown by red square in the figure) will give the output shown by red which is 1×1×4.
- The first row of the output (8 positions) will correspond to 8, 14×14 squares obtained by a stride of 2 (3 squares are shown on the input image).
- Then a vertical stride of 2 will be taken, followed by horizontal strides of 2 to slide the 14×14 region to generate the second row of the output etc.

Convolution implementation of sliding windows





Sliding windows implementation will handle each 14×14 region one by one.



Convolutional implementatio will handle all squares at once by one forward pa

- Weakness of the above algorithm: The positions of the bounding boxes is not going to be too accurate.

Andrew Ng

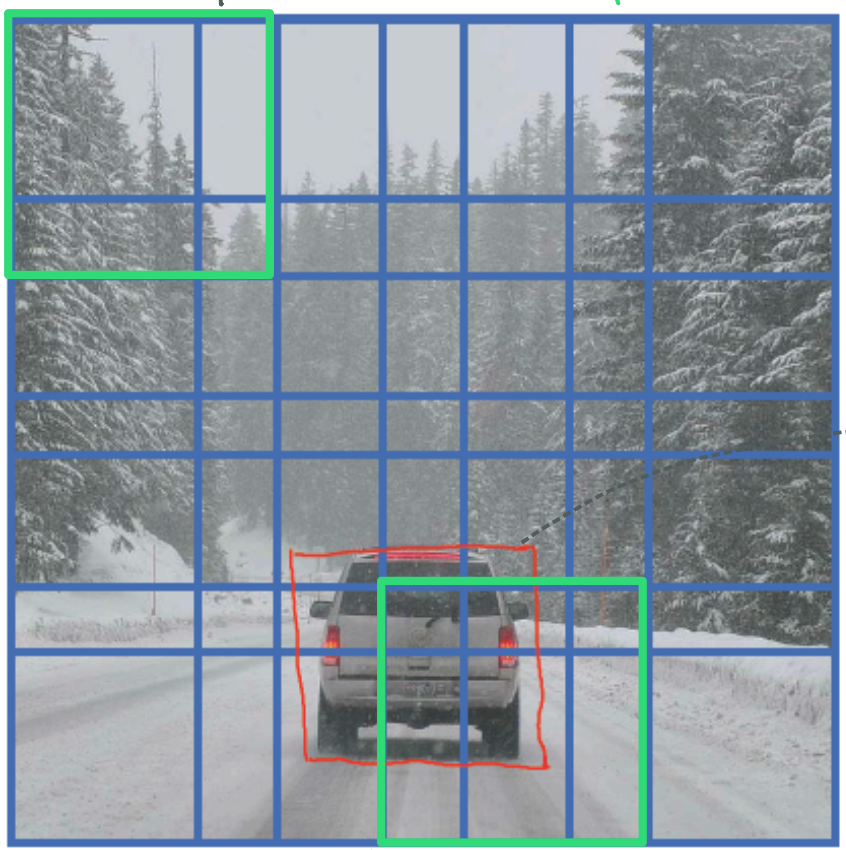
deeplearning.ai

Object Detection

Bounding box predictions

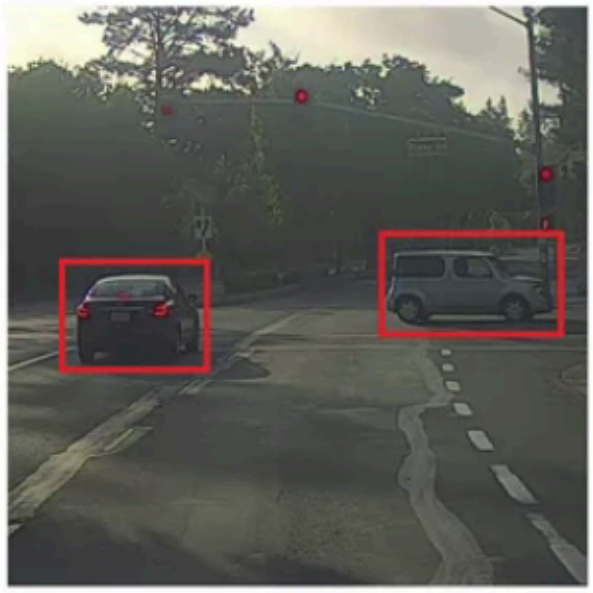
- The convolutional implementation of sliding windows can not output the most accurate bounding boxes. Is there a way to get this algorithm output more accurate bounding boxes?

Output accurate bounding boxes



Andrew Ng

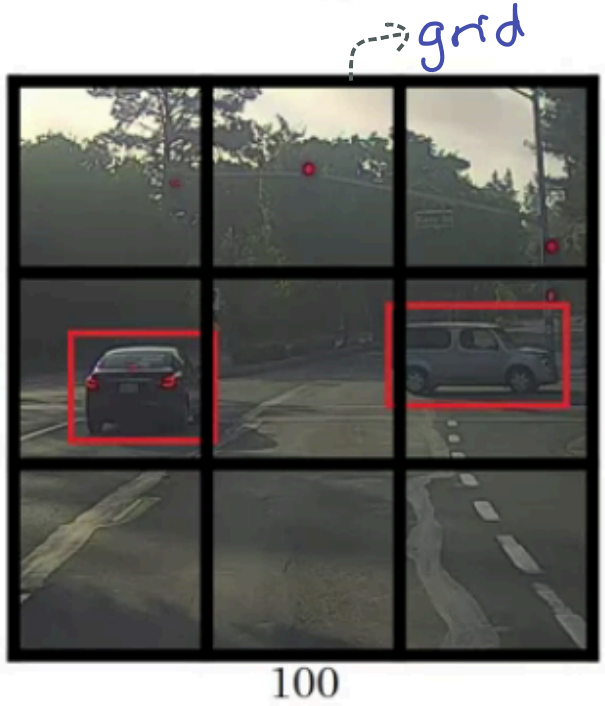
YOLO algorithm



- A good way to get the sliding windows algorithm output more accurate bounding boxes is the YOLO algorithm.
- YOLO: You Only Look Once

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

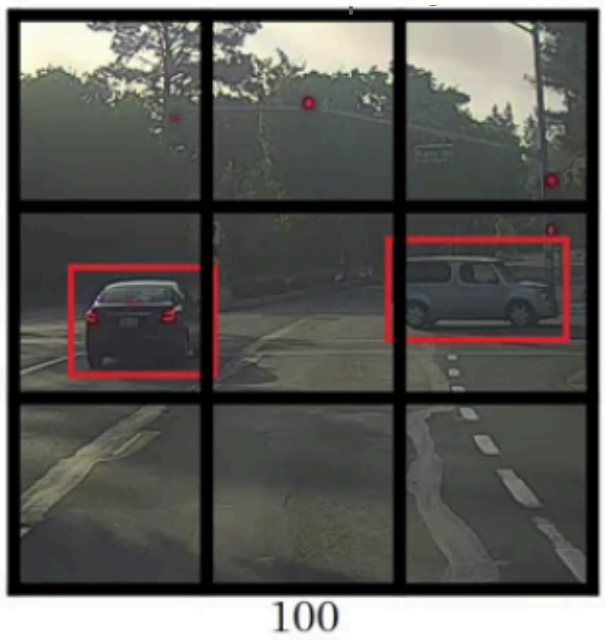
YOLO algorithm



- The input image is 100x100
- Place a grid on the image.
- For simplicity let the grid be 3x3. (in actual implementation a finer grid e.g. 19x19 may be used)
- Basic idea: Apply the image classification and localization algorithm to each of the 9 grid cells in the image.

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

YOLO algorithm



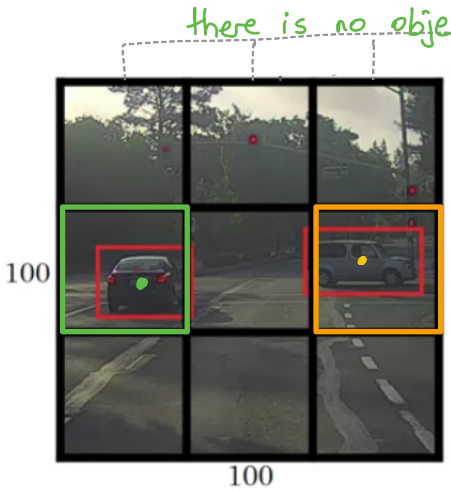
Labels for training are defined as follows:
For each grid cell:

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

p_c → is there an object
b_x, b_y, b_h, b_w → bounding box
c_1 → pedestrian
c_2 → car
c_3 → motorcycle

we have this vector for each of the grid cells

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]



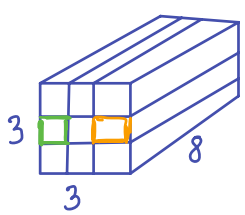
$$y = \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix} \quad (1)$$

- The image has two objects (cars)
- YOLO algorithm takes the midpoint of each of the objects and then assigns the object to the grid cell containing the midpoint (the green and yellow dots in the above image are the midpoints.
- The central grid cell contains some parts of both cars, but we'll pretend the central grid has no interesting object so that the target vector is the same as (1).
- The target label for the green cell in the figure (also the yellow cell)

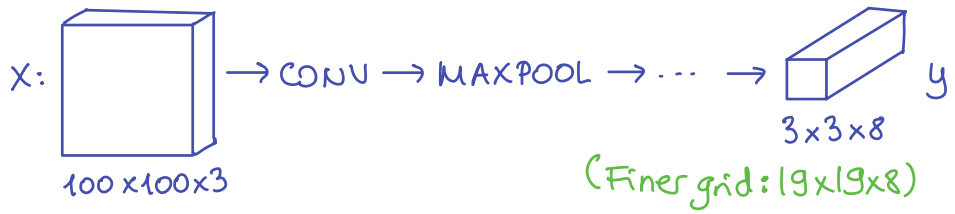
$$y = \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}_{8\times1} \qquad y = \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}_{8\times1}$$

- For each of the 9 grid cells, we have such a 8x1 target y vector.
- The target output for all 9 cells will be: 3x3x8

Target output volume:



- To train the NN use backpropagation to map x to y:

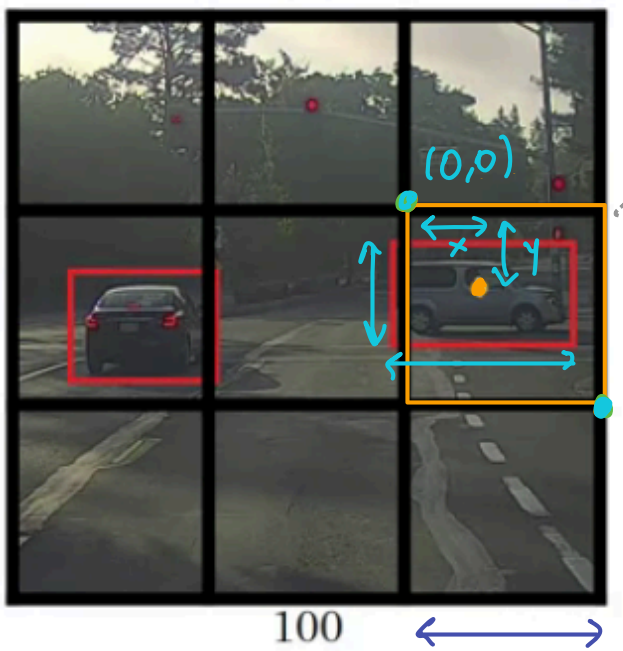


- Advantage of YOLO: The algorithm outputs precise bounding boxes with any aspect ratio, and precise coordinates that aren't dictated by the stride size of the sliding window classifier.
- As long as you don't have more than 1 object in the grid cell, this algorithm should work OK. (Multiple obj. case will be addressed later)
- Using a finer grid reduces the chance that there are multiple objects in the same grid cell.

- This is a convolutional implementation and we're not running the same algorithm 9 times for each cell. We use 1 convnet with shared computations, which makes YOLO an efficient algorithm.
- YOLO can be used for real-time object detection since convolutional implementation runs really fast.

- Next: How do we encode the bounding boxes: b_x, b_y, b_h, b_w?

Specify the bounding boxes



$$y = \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

- How do we specify the bounding box?
- The relative position of the centroid of the car (orange point) with respect to the upper left corner is used to define b_x, b_y.
- The height and width is defined relative to the size of the grid cell. e.g. the width of the red box is approximately 90% of the width of the grid cell.

between 0 and 1: $b_x \simeq 0.4$, $b_y \simeq 0.3$, $b_w \simeq 0.9$, $b_h \simeq 0.5$ } could be >1

Since orange pt. is inside the grid cell.

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Andrew Ng

- Next: How can we make this algorithm work even better?

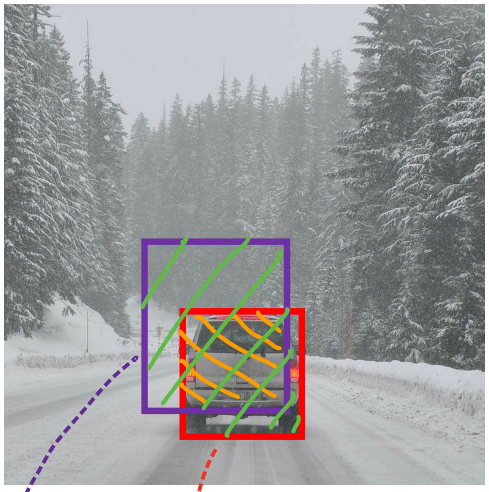
deeplearning.ai

Object Detection

Intersection over union (IoU)

- How do we tell if the object detection algorithm is working well?
- IoU is used to evaluate the object detection algorithm and also to add another component to the object detection algorithm to make it work better.

Evaluating object localization



Ground truth bounding box

Estimated bounding box

Intersection over Union (IoU)

$$= \frac{\text{size of intersection}}{\text{size of union}}$$

- If two bounding boxes are perfectly the same $\Rightarrow$ IoU = 1

"Correct" if IoU ≥ 0.5 (result is acceptable)

0.6 (to be more strict)

0.7

More generally, IoU is a measure of the overlap between two bounding boxes.

Andrew Ng

deeplearning.ai

Object Detection

Non-max suppression

- A problem of the object detection algorithm we have learned so far: the algorithm may find multiple detections of the same objects.
- Non-max suppression is a way to make sure that the algorithm detects each object only once.

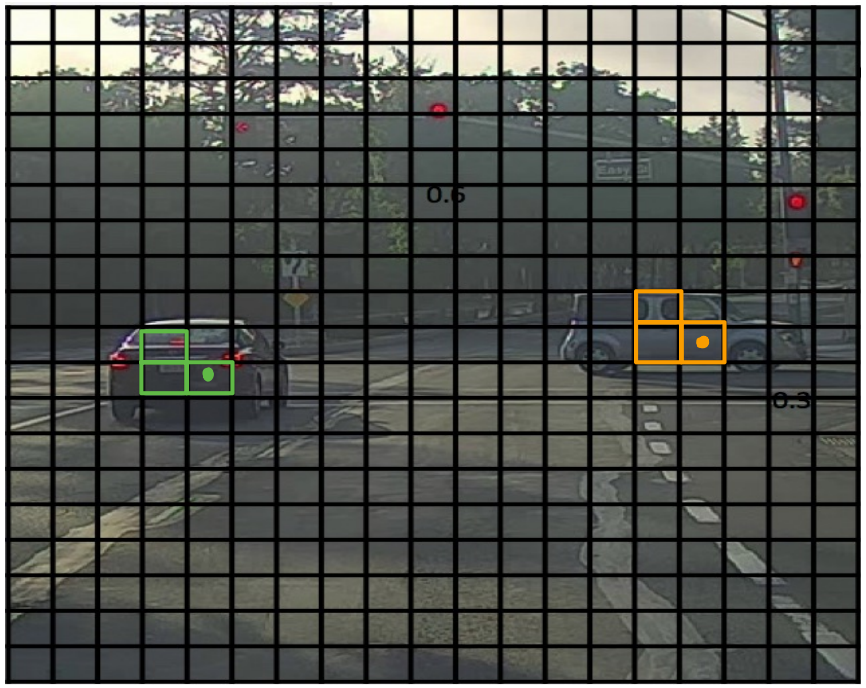
Non-max suppression example

- Goal: Detect pedestrians, cars and motorcycles in the image.

Andrew Ng

Non-max suppression example

- Place a 19x19 grid on the image.
- Each car has 1 midpoint that is assigned to 1 grid cell.
- We run an object detection and localization algorithm for each cell.
- It is possible that multiple grid cells might detect a car in it. (The grid cells shown with yellow or green may all think there's a car in them.

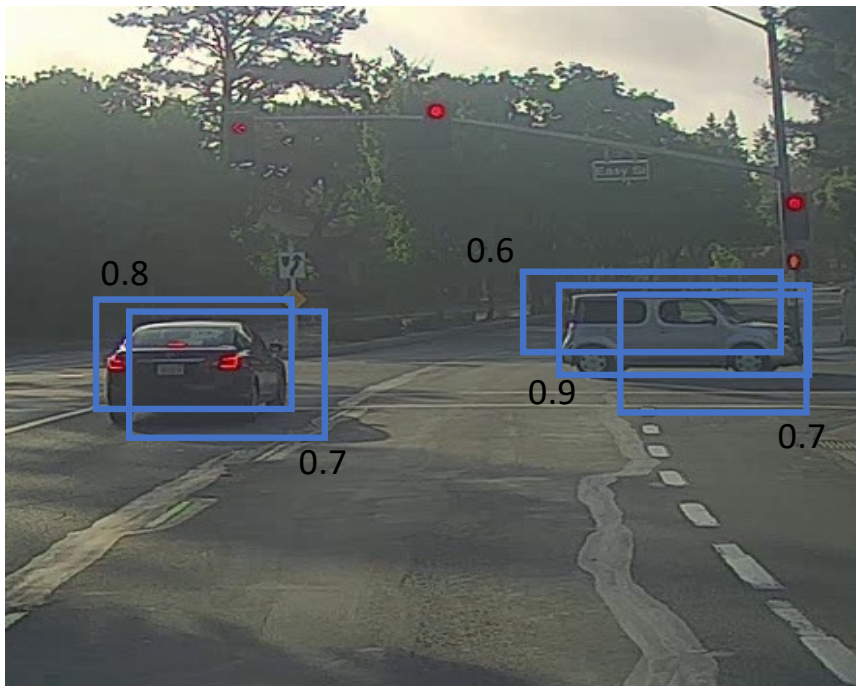


19 x 19

Andrew Ng

Non-max suppression example

- Algorithm might detect $p_c = 1$ for the blue boxes.

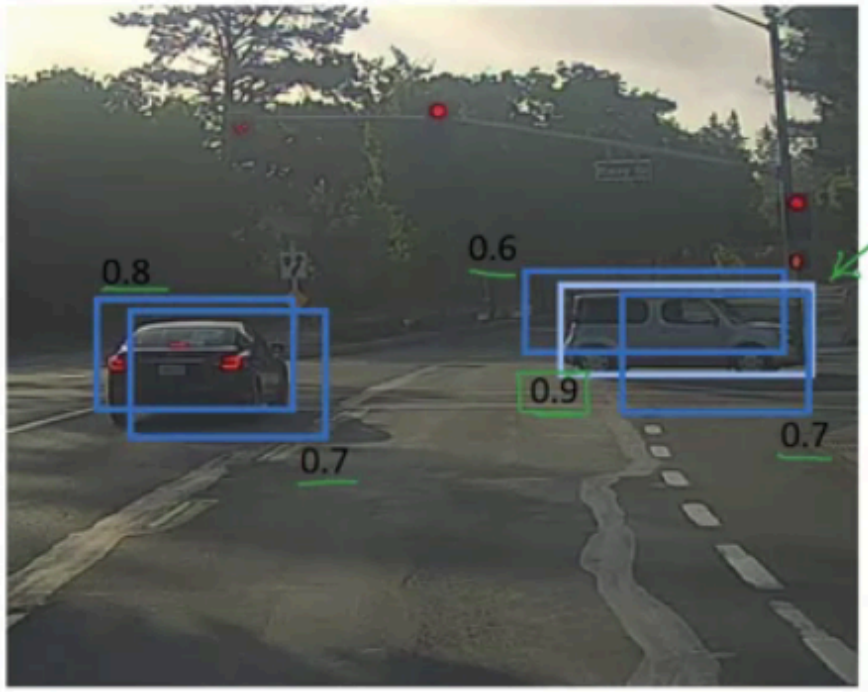


- Non-max suppression cleans up these detections so that we end up with one detection per car.

Andrew Ng

Non-max suppression example

- It looks at the probabilities associated with each of these detections.



- It first takes the largest one (i.e. 0.9 in the figure)

p_c

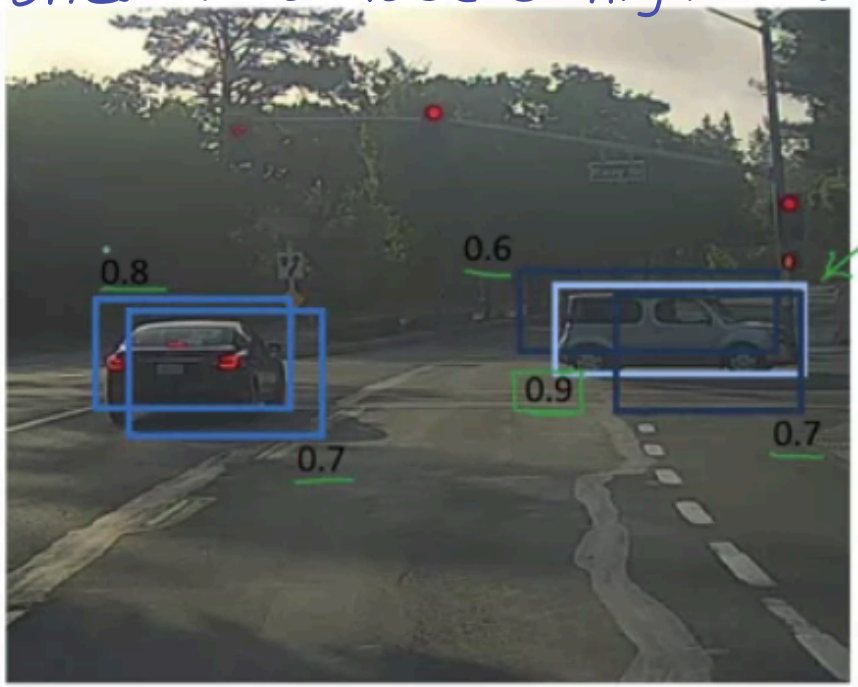
- It then highlights the most confident detection, and says "the car is detected here." (box shown with light blue color)

Andrew Ng

Andrew Ng

Non-max suppression example

- Non-max suppression then looks at all the remaining rectangles and suppresses the ones that have a high IoU with the highlighted box.



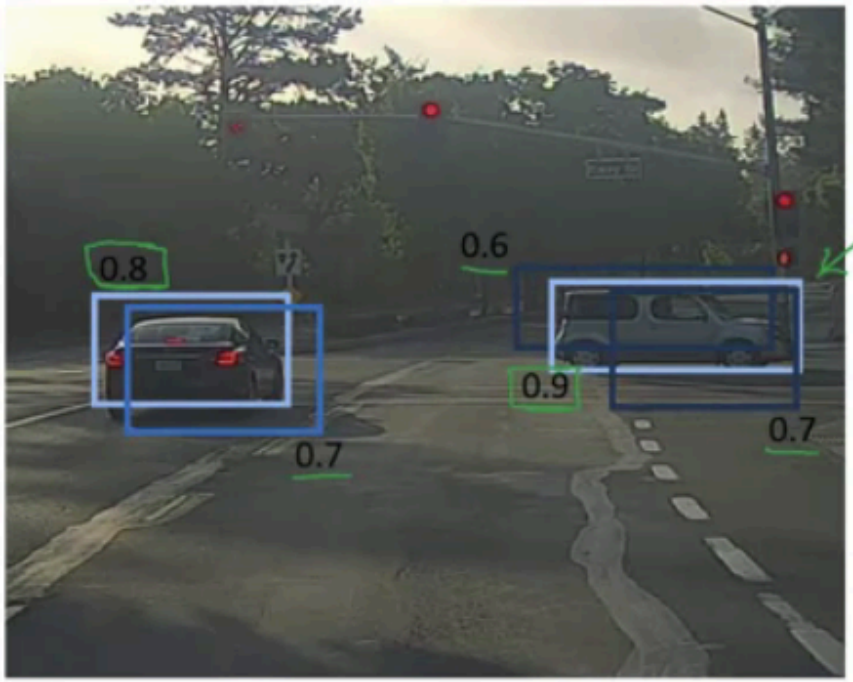
Pc

- The suppressed boxes are shown with dark blue color.

Andrew Ng

Non-max suppression example

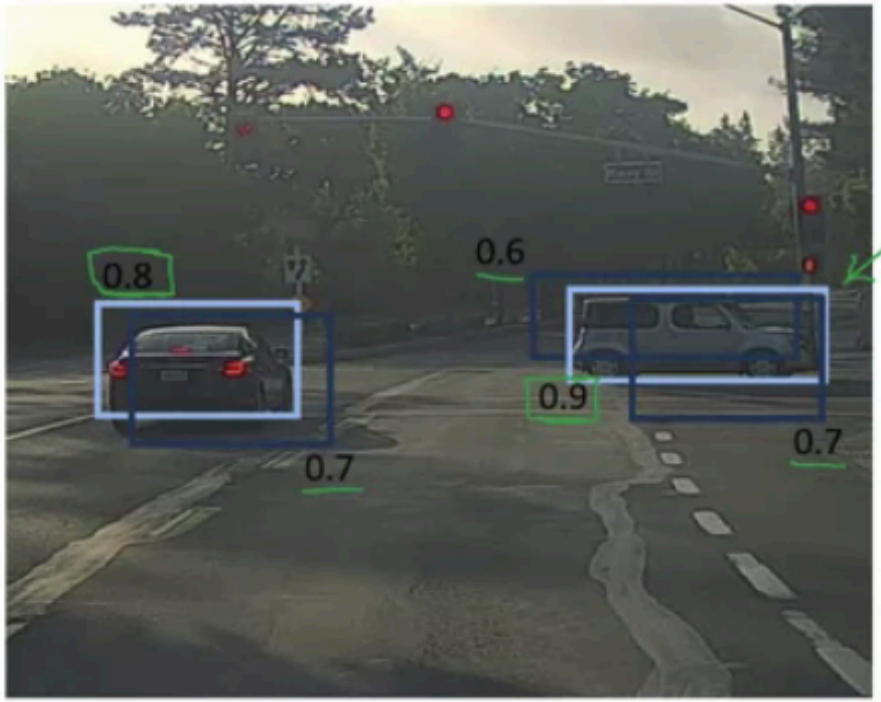
- Next, we go through all the remaining rectangles and find the one with the highest probability. (0.8 on the left)
- It highlights the box as "a car is detected here".



Andrew Ng

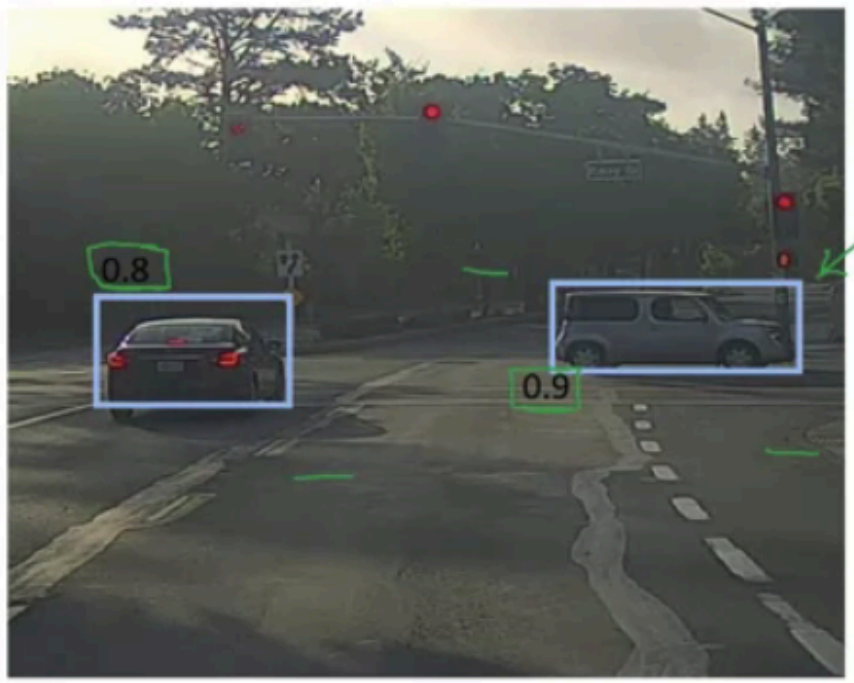
Non-max suppression example

- Then, the boxes that have a high IoU with the last highlighted rectangle are suppressed (shown with dark blue on the left)



p_c

Non-max suppression example



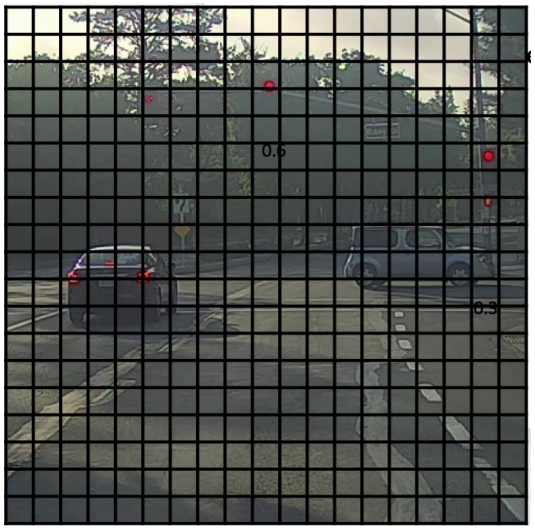
p_c

- The suppressed boxes are removed.
- The detected two cars are shown in the figure.

Andrew Ng

Details of the algorithm: (For one object: car)

Non-max suppression algorithm



19×19

Each output prediction is: $\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \end{bmatrix}_{5\times 1}$

(for car detection on each $19 \times 19 = 381$ cells)

Discard all boxes with $p_c \leq 0.6$

While there are any remaining boxes:

- Pick the box with the largest p_c
  Output that as a prediction.
- Discard any remaining box with IoU ≥ 0.5 with the box output in the previous step

Andrew Ng

- For the 19x19 grid in the figure above, we will get a 19x19x8 volume as output.
- Let's say we're doing only car detection here, so c_1, c_2, c_3 are not used, so the output volume will be 19x19x5.
- For each grid cell, we output p_c (prob. that there's a car) and the bounding box (4 numbers).
- To implement non-max suppression:
  - We can first discard all boxes with p_c less than a threshold, e.g. $p_c \leq 0.6$

- If you try to detect three objects: pedestrians, cars, and motorcycles, the output vector will have 3 additional components (c_1, c_2, c_3)
- Then, carry out non-max suppression 3 times, one on each of the output classes.

- Next: The idea of anchor boxes makes the YOLO algorithm work much better.

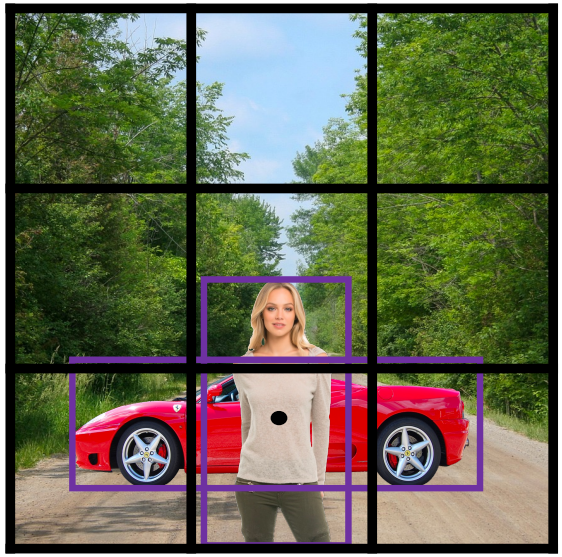
deeplearning.ai

Object Detection

Anchor boxes

- One problem with the object detection as you've seen so far is that each of the grid cells can detect only one object.
- What if a grid cell wants to detect multiple objects?
- You can use the idea of anchor boxes.

Overlapping objects:



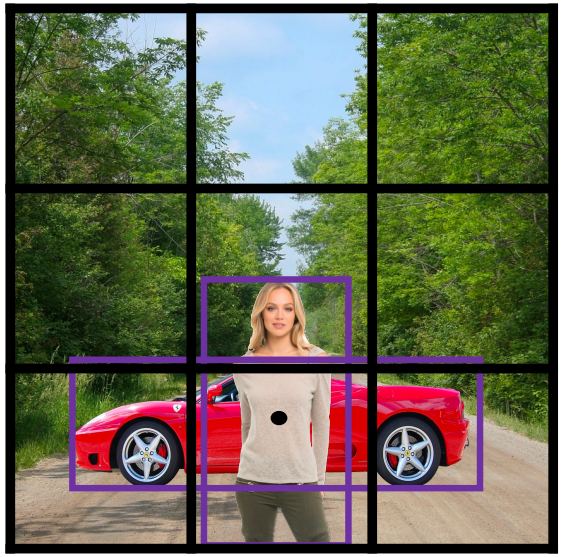
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

- In the image, which has a 3x3 grid, the midpoint of the pedestrian and the midpoint of the car are almost in the same place, and both of them fall into the same grid cell.
- For that grid cell, y outputs this vector where you are detecting 3 classes: pedestrians, cars and motorcycles.
- It won't be able to output 2 detections. So, we have to pick one of the two detections to output.

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

- With the idea of anchor boxes, we are going to define two different shapes called anchor boxes or anchor box shapes. We are going to associate two predictions with the two anchor boxes. (May use more anchor boxes)

Overlapping objects:



(pedestrian)
Anchor box 1:

(car)
Anchor box 2:

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

class label: y=
(repeat this twice)

p_c b_x b_y b_h b_w c_1 c_2 c_3 → anchor box 1

p_c b_x ... c_3 → anchor box 2

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Andrew Ng

- Pedestrian has a bounding box which is more similar to anchor box 1, so use it for the pedestrian object.

Anchor box algorithm

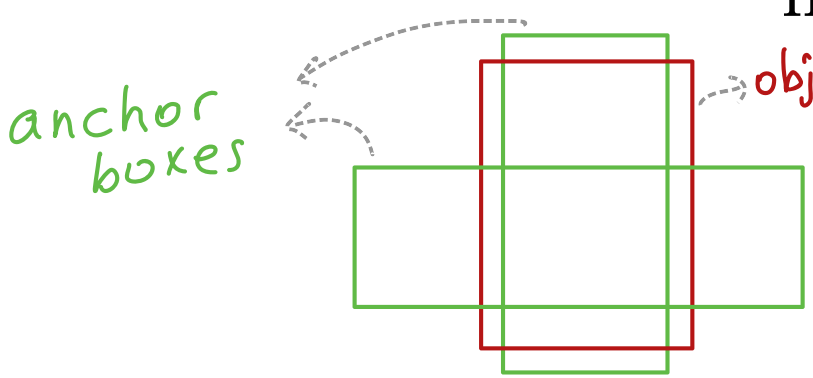
Previously:

Each object in training image is assigned to grid cell that contains that object's midpoint.

Output y: $3 \times 3 \times 8$ ↳ if 3 object classes

With two anchor boxes:

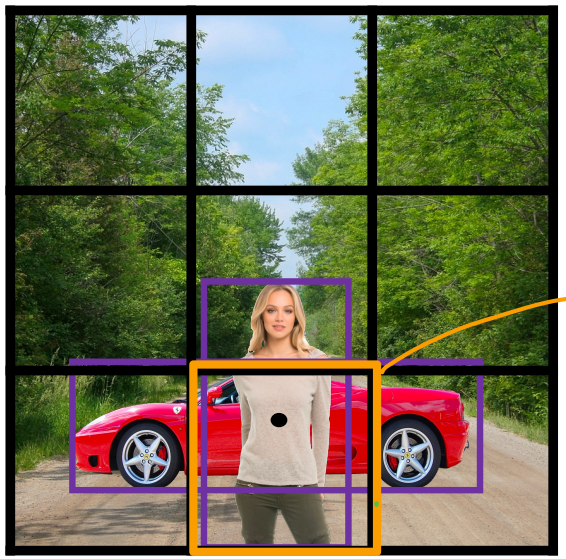
Each object in training image is assigned to grid cell that contains object's midpoint and anchor box for the grid cell with highest IoU.



- See which of the two anchor boxes has a higher IoU with the ground truth bounding box.
- The object gets assigned not just to a grid cell but to a pair: (grid cell, anchor box)

Andrew Ng

Anchor box example



for this cell

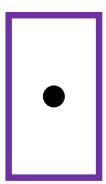
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \begin{bmatrix} 1 \\ bx \\ by \\ bh \\ bw \\ 1 \\ 0 \\ 0 \\ 1 \\ bx \\ by \\ bh \\ bw \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

- Pedestrian has a shape more similar to anchor box 1.
- Assign the pedestrian to the top half of the vector.

Bounding box of the car

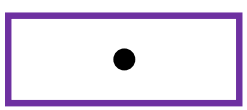
→ car

Anchor box 1:



pedestrian is more similar to anchor box 1

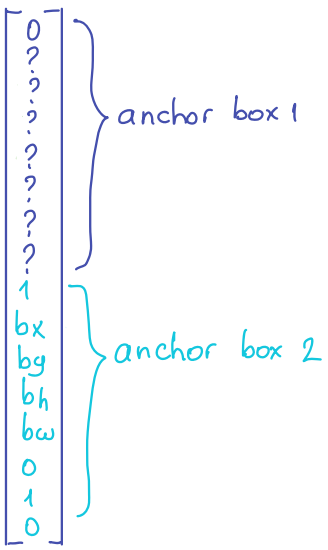
Anchor box 2:



shape of the car is more similar to anchor box 2

Andrew Ng

- What if the yellow grid cell in the above figure only had a car?
  → The shape of the bounding box of the car is still more similar to anchor box 2, the second half of y would be containing car information:



- What if you have two anchor boxes but 3 objects in the same grid cell? → That's one case the algorithm doesn't handle well. It will just implement some default tie-braker in that case.
- What if you have two objects associated with the same grid cell, but both of them have the same anchor box shape?
  → That's another case the algorithm doesn't handle well.
  → May implement a default tiebraker.
  → Hopefully it won't happen often, and won't influence the performance much.
- Anchor boxes are useful if there are multiple objects in the some grid cell. In practice, that happens rarely, especially if we use a 19x19 grid, rather than a 3x3 grid. The chance of two objects having the same midpoint in 381 cells is quite small.
- Anchor boxes allows your learning algorithm to specialize better. e.g. if your dataset has some tall, skinny objects like pedestrians,

and some white objects like cars, this allows the learning algorithm to specialize so that some of the output units can specialize in detecting tall, skinny objects like pedestrians and some of the output units can specialize in detecting white, fat objects like cars.

- How do we choose the anchor boxes?
  - → Choose them by hand or
  - → choose 5-10 anchor box shapes that covers the types of objects you detect.
  - → May use K-means algorithm to group together the shapes of objects you detect. Then use several representatives of each cluster to determine the anchor box shapes.

deeplearning.ai

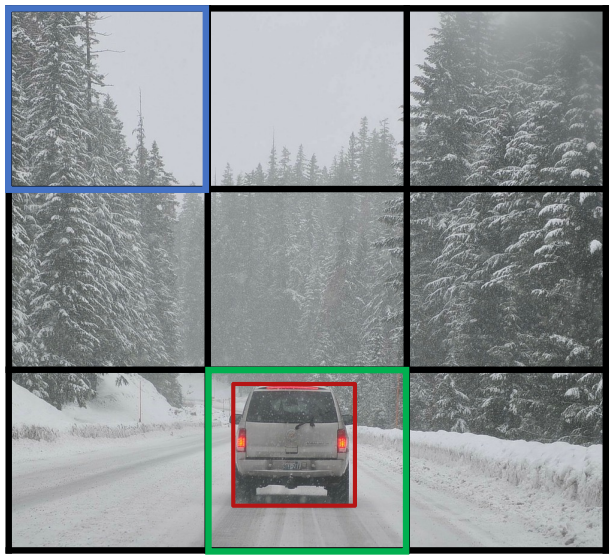
Object Detection

Putting it together: YOLO algorithm

Goal: We want to detect 3 objects and we are using 2 anchor boxes.

Training

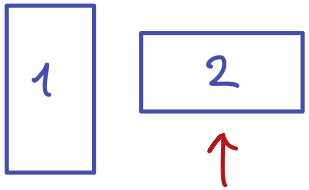
1 - pedestrian
2 - car
3 - motorcycle



$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \quad \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix} \quad \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

8

Anchor boxes



car with red bounding is wider than taller

3×3×16

y is 3×3×2×8

↓ #anchors

↳ 5+#classes

for blue cell in the Figure (no object in it)

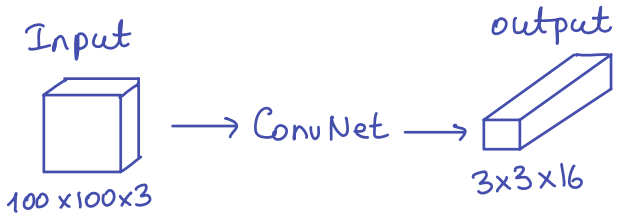
Green cell in the figure.

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

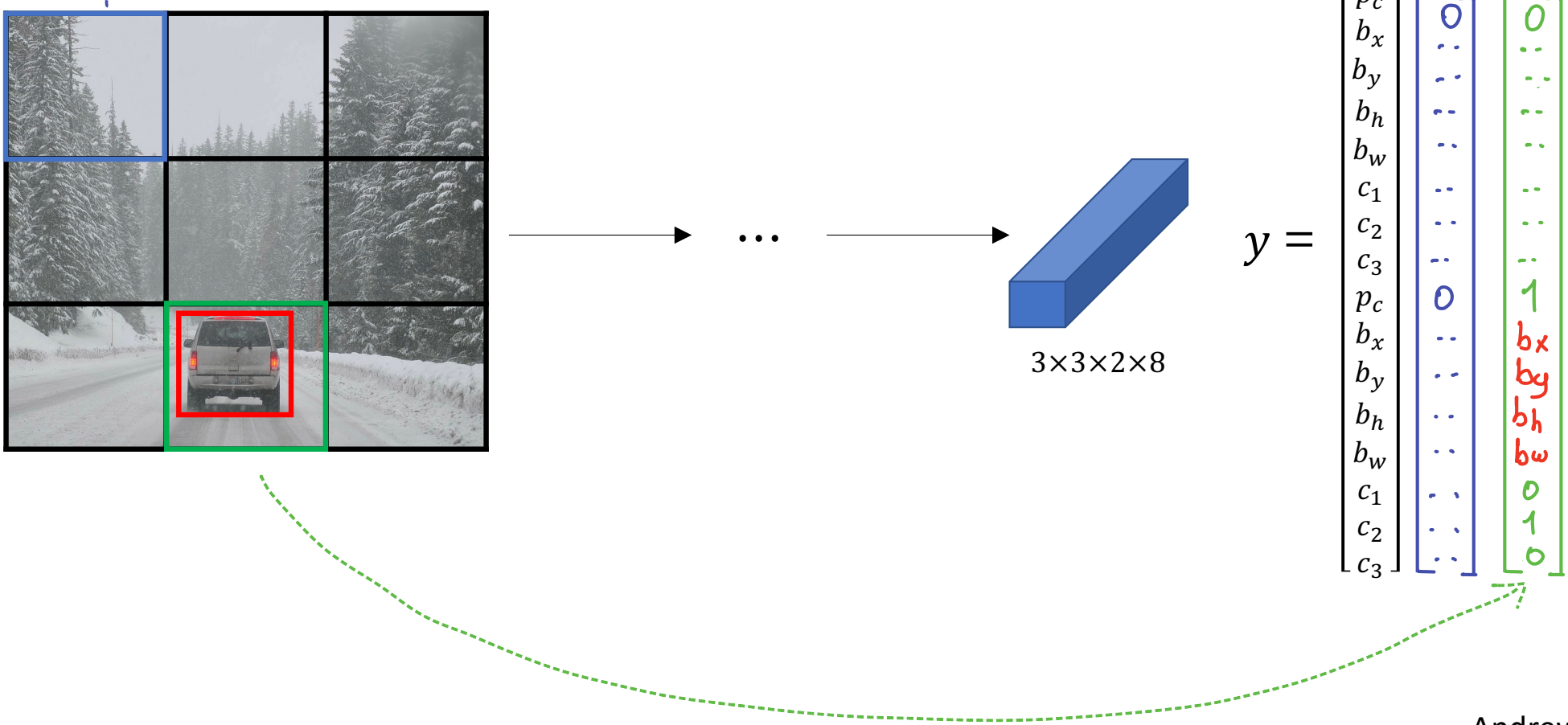
Andrew Ng

- The bounding box of the car has a slightly higher IoU with anchor box 2. So, the information of the car is written in the lower half of the y vector.
- For each of the 9 grid cells the output vector is 16x1, Hence, the output volume is 3x3x16 dimensional.
    - In practice the output volume could be:
        - 19x19x16 (for a 19x19 grid cell and 2 anchor boxes)
        - or 19x19x40 (for a 19x19 grid cell and 5 anchor boxes)

- The training process:



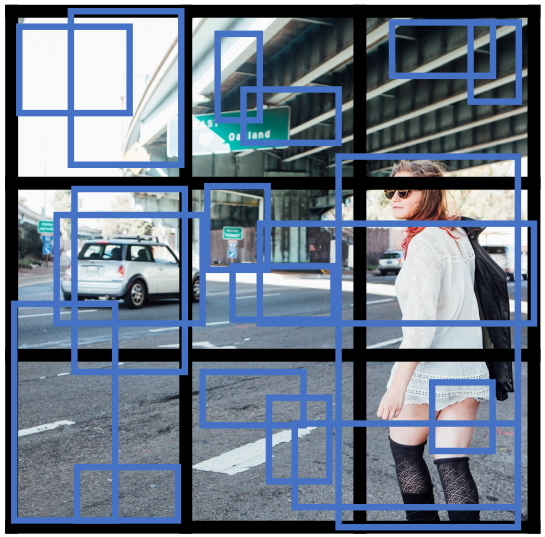
Making predictions



Andrew Ng

Outputting the non-max supressed outputs

If we're using 2 anchor boxes:



- For each grid call, get 2 predicted bounding boxes.
- Get rid of low probability predictions.
- For each class (pedestrian, car, motorcycle) use non-max suppression to generate final predictions.

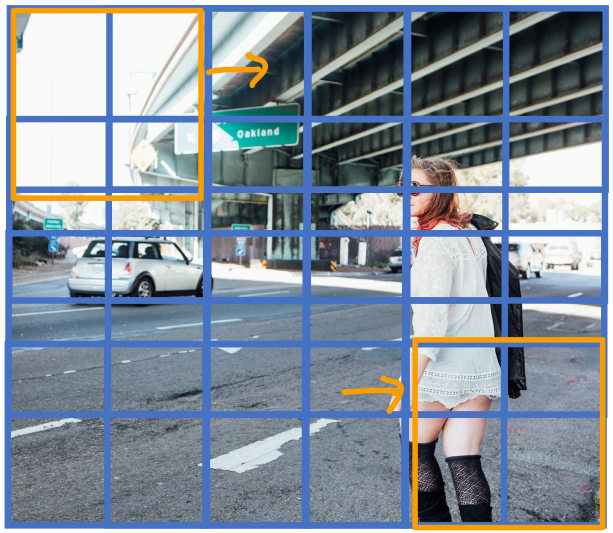
Andrew Ng

deeplearning.ai

Object Detection

Region proposals (Optional)

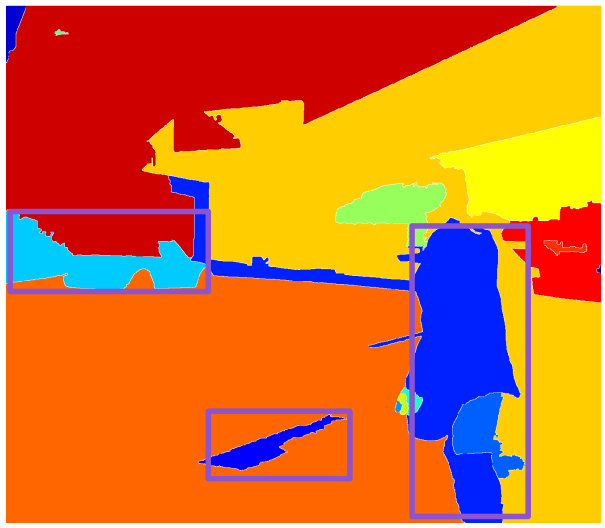
Region proposal: R-CNN (Regions with CNNs)



Sliding windows idea:
Take a trained classifier and run it across all windows to detect if there's a pedestrian, car or motorcycle.

You could run the algorithm convolutionally, but it classifies a lot of regions where there's no object.

Segmentation Algorithm
~2000 blobs



R-CNN Idea: Pick just a few regions that makes sense to run your classifier on.
- Segment the image first.
- Put a bounding box around the blobs and classify the "blob".
- Quite slow

[Girshik et. al, 2013, Rich feature hierarchies for accurate object detection and semantic segmentation]

Faster algorithms

R-CNN: Propose regions. Classify proposed regions one at a time. Output label + bounding box.

more accurate than the bounding box of segmentation algorithm.

Fast R-CNN: Propose regions. Use convolution implementation of sliding windows to classify all the proposed regions.

Faster R-CNN: Use convolutional network to propose regions.

[Girshik et. al, 2013. Rich feature hierarchies for accurate object detection and semantic segmentation]
[Girshik, 2015. Fast R-CNN]
[Ren et. al, 2016. Faster R-CNN: Towards real-time object detection with region proposal networks]

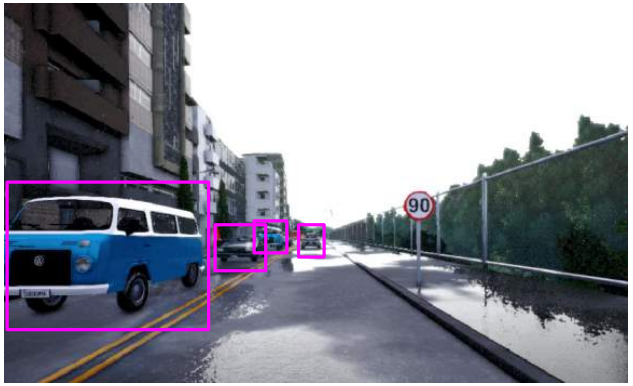
deeplearning.ai

Convolutional Neural Networks

Semantic segmentation with U-Net

Object Detection vs. Semantic Segmentation

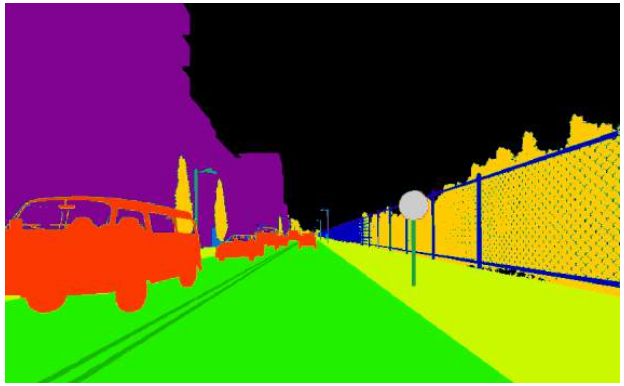
Input image



Object Detection

- Draw bounding boxes around cars.
- May be sufficient for self-driving car.

(Not meaningful to draw a bounding box around the road.)



Semantic Segmentation

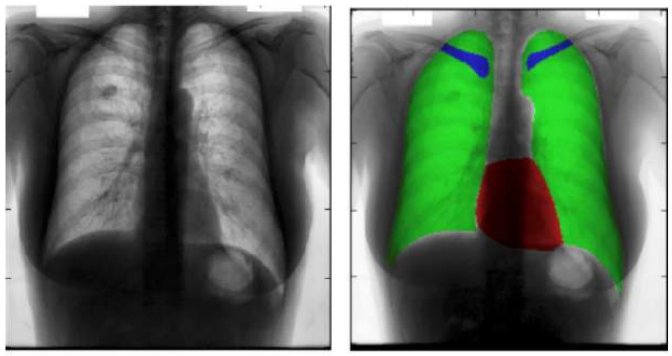
Goal: To label every single pixel.

- More accurate for self-driving car.

Andrew Ng

Motivation for U-Net

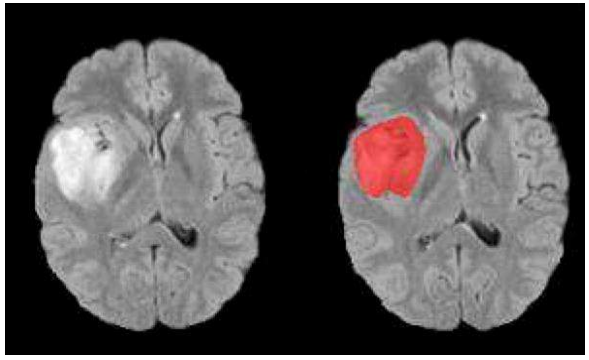
Segment the lungs, heart and collor bones.



Chest X-Ray

Easier to make a diagnosis or plan a surgery.

Segment the tumor automatically.

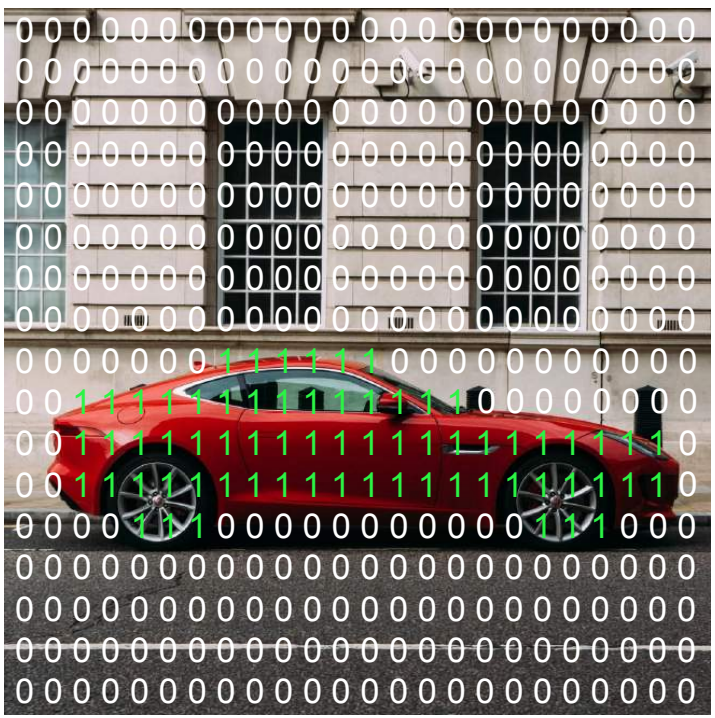


Brain MRI

- Radiologists save time

[Novikov et al., 2017, Fully Convolutional Architectures for Multi-Class Segmentation in Chest Radiographs]
[Dong et al., 2017, Automatic Brain Tumor Detection and Segmentation Using U-Net Based Fully Convolutional Networks]

Per-pixel class labels

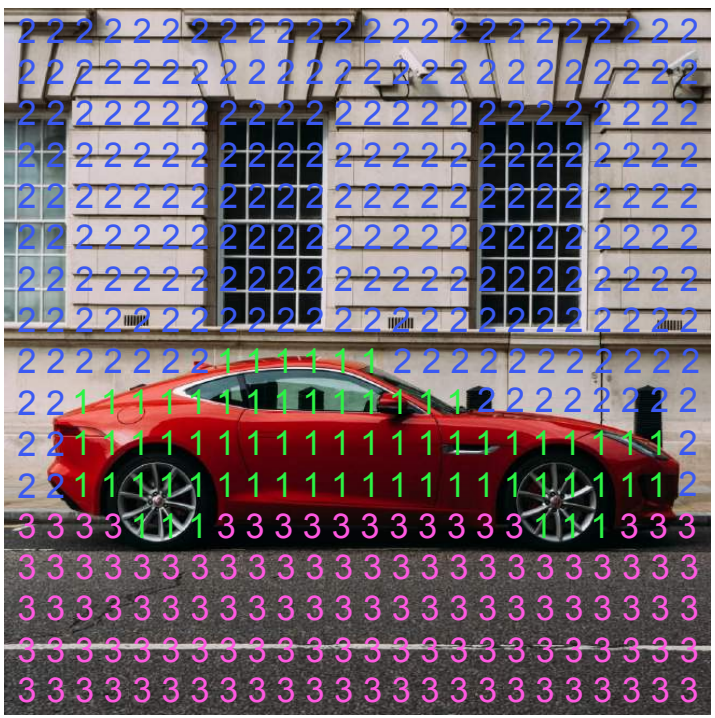


Goal: Segment the car only.

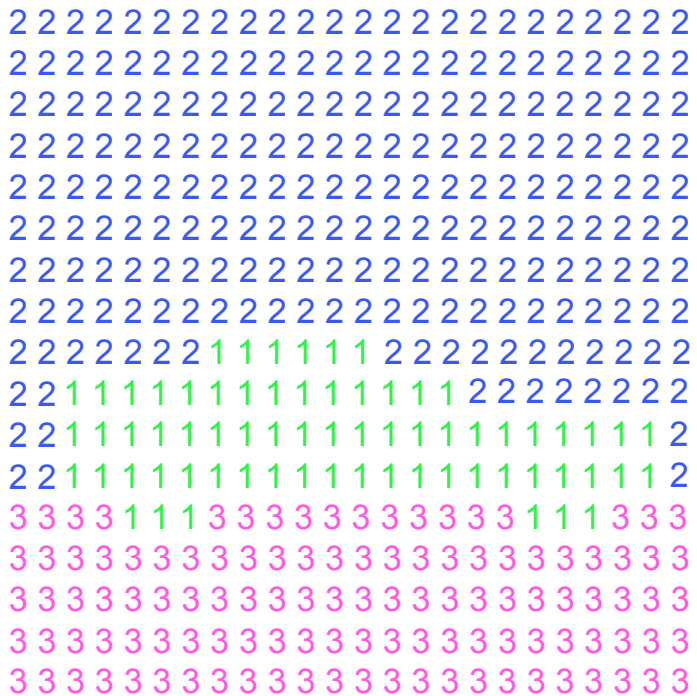
- Segmentation algorithm will output 1 or 0 in the image.

1. Car
0. Not Car

Per-pixel class labels



1. Car
2. Building
3. Road



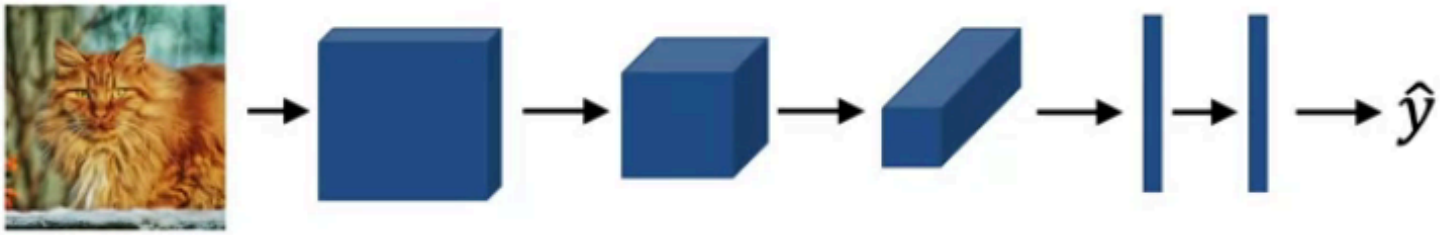
Segmentation Map

The output we would like the U-net algorithm train to give

Andrew Ng

Deep Learning for Semantic Segmentation

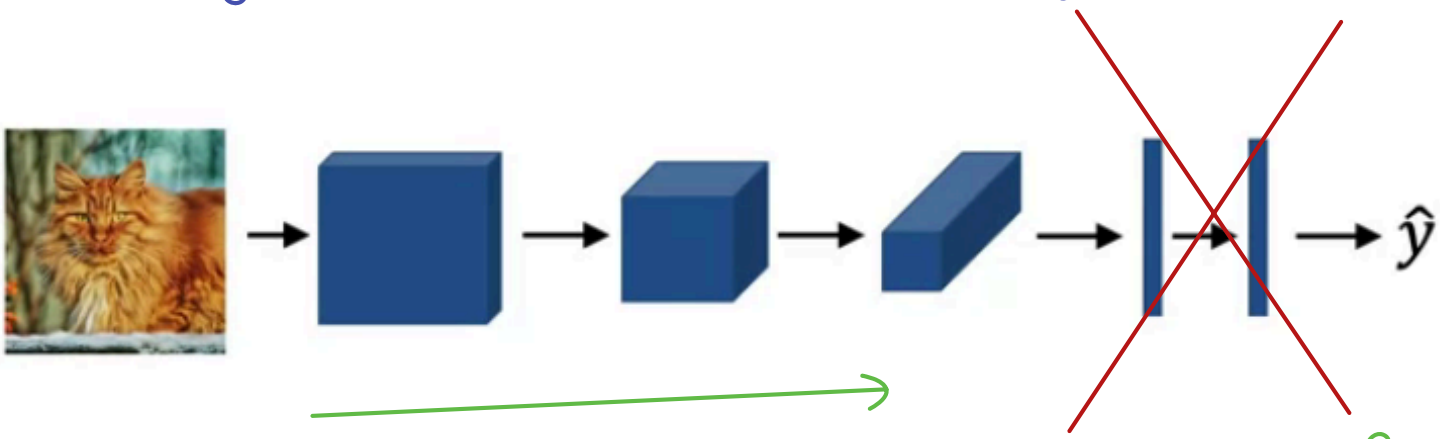
Start with object recognition neural network architecture



Andrew Ng
Andrew Ng

Deep Learning for Semantic Segmentation

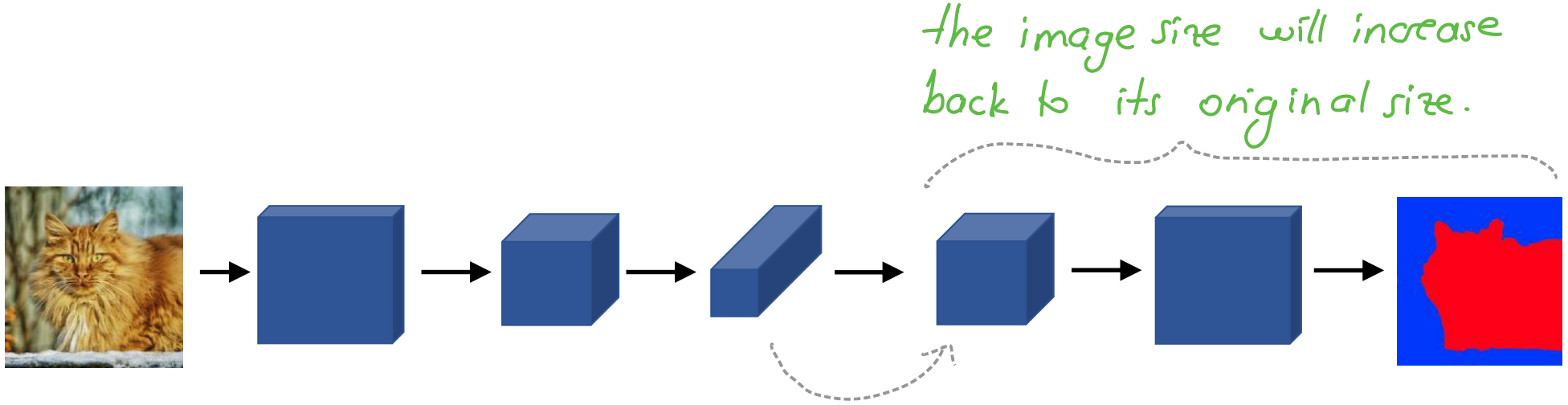
First, get rid of the final few layers.



- Image size gets smaller as we go from left to right.
- It needs to get bigger so that the output will be a full-size image of pixel labels.

Andrew Ng
Andrew Ng

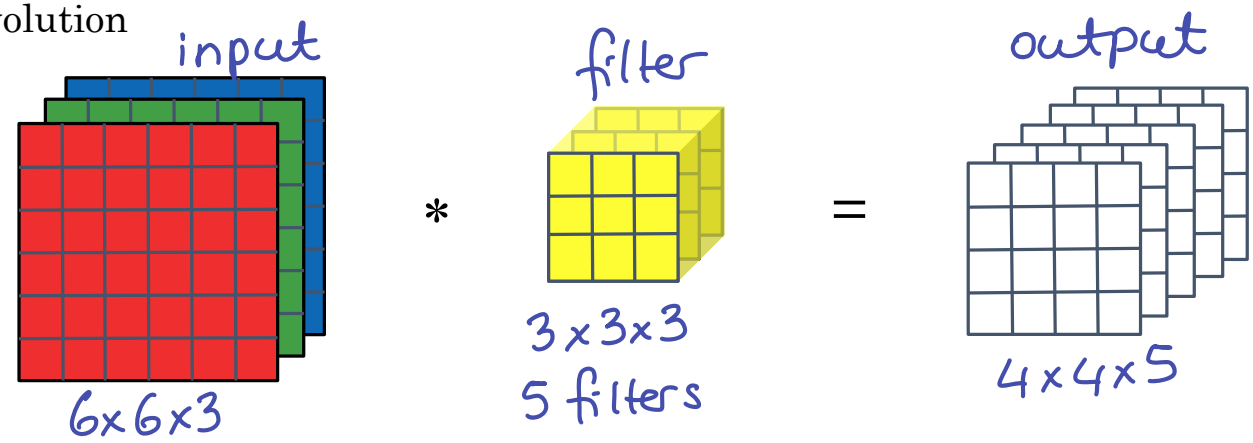
Deep Learning for Semantic Segmentation



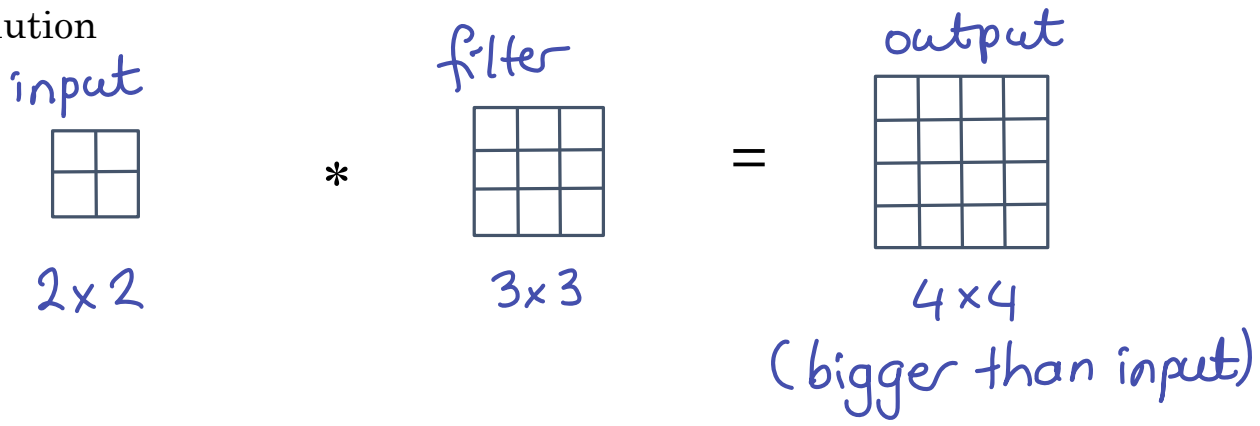
Andrew Ng

Transpose Convolution

Normal Convolution

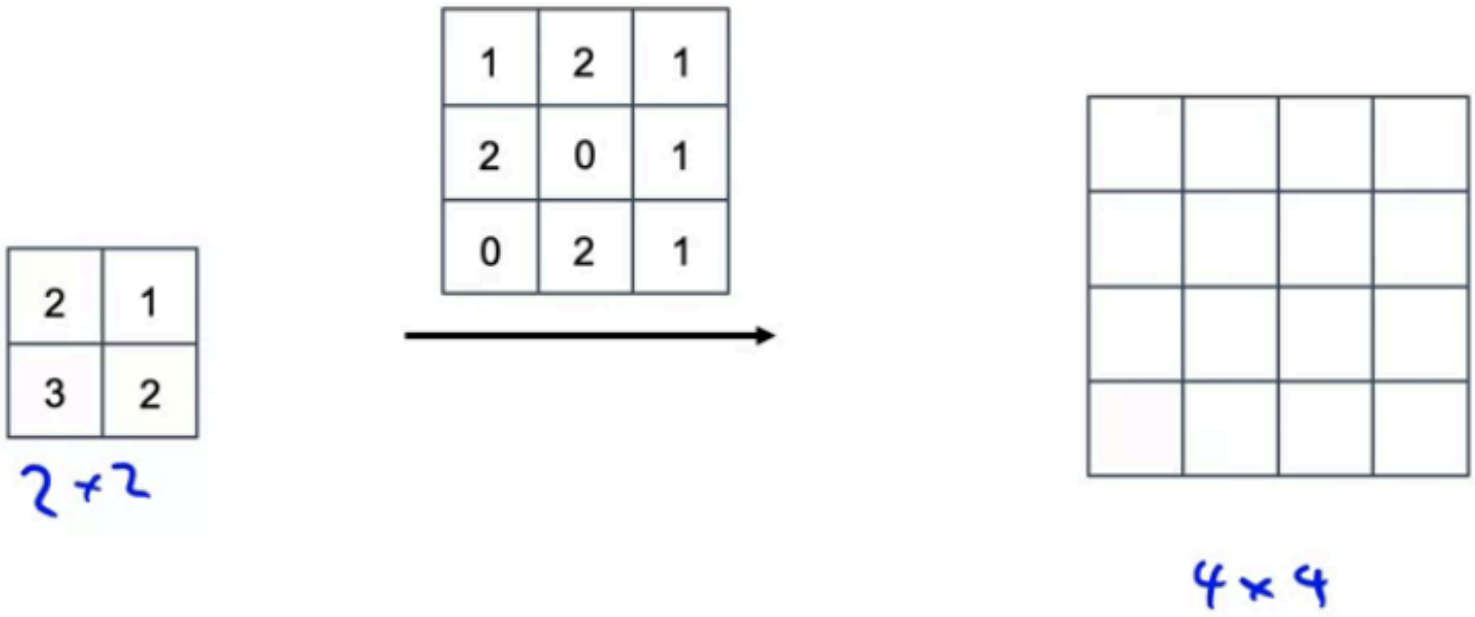


Transpose Convolution



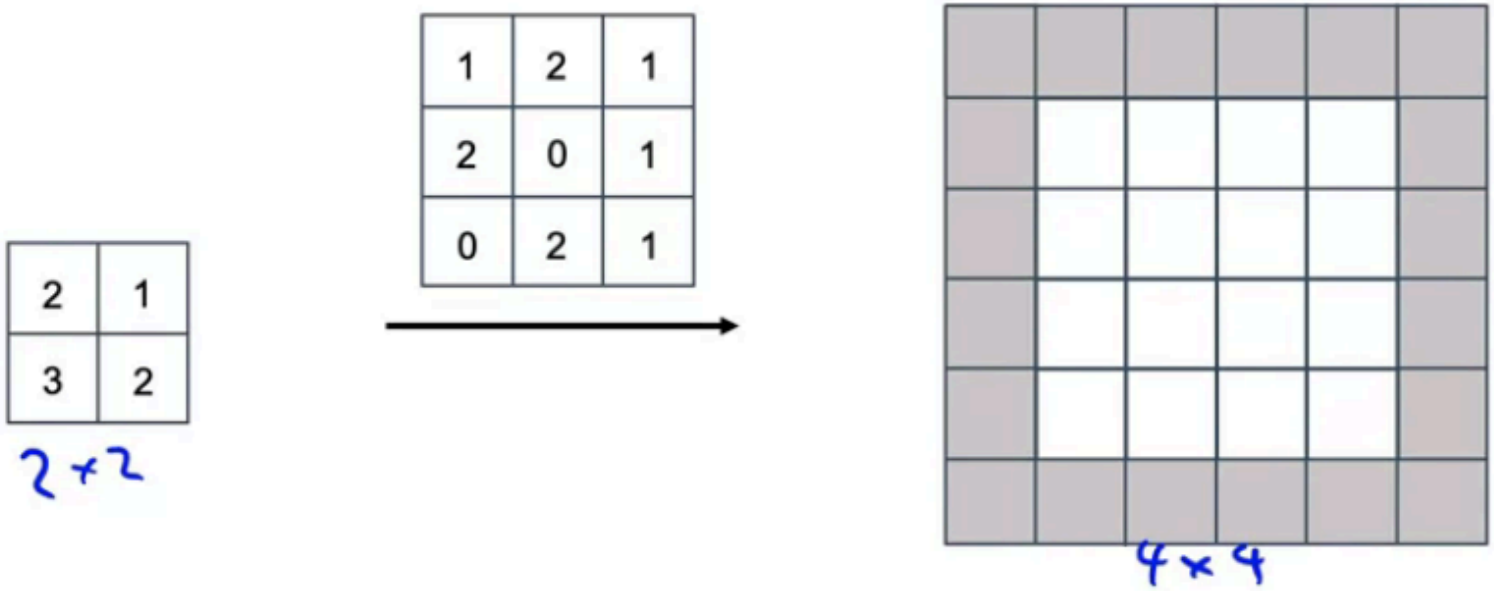
Andrew Ng

Transpose Convolution



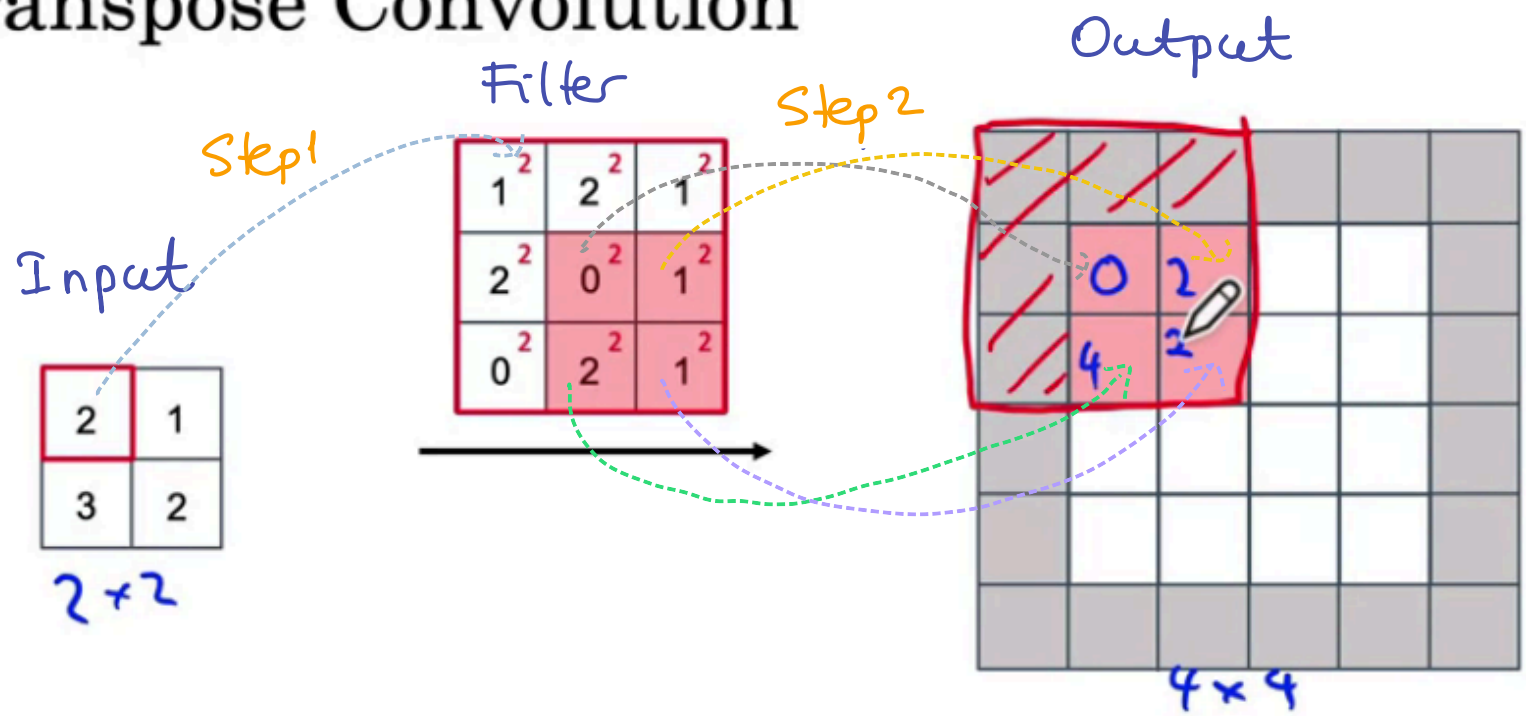
Andrew Ng

Transpose Convolution



Andrew Ng

Transpose Convolution



filter $f \times f = 3 \times 3$ padding $p = 1$ stride $s = 2$

Andrew Ng

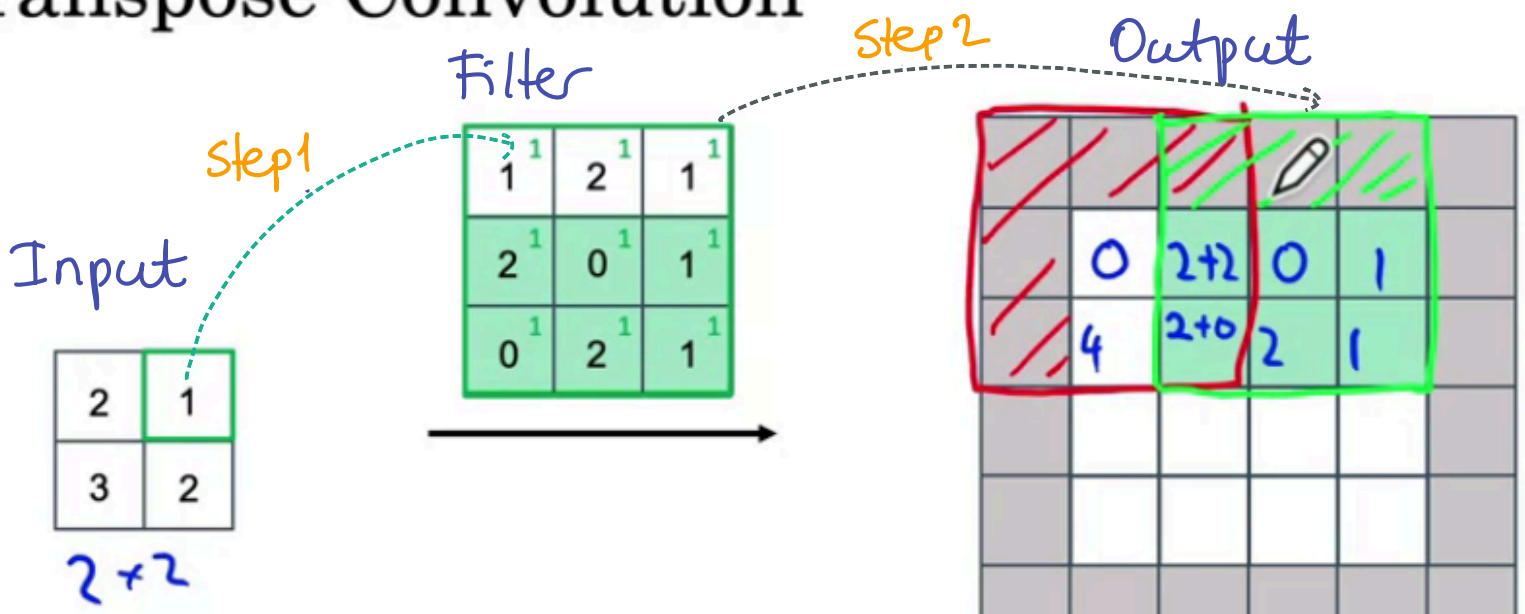
- In the regular convolution, you would take the filter and place it on top of the inputs and then multiply and sum up.
- In the transpose convolution, instead of placing the filter on the input, you would place a filter on the output.

Steps of transpose convolution (in the above slide)

Step1
- Take the 2 in the input (red 1x1 square)
- Multiply the filter coefficients by 2

Step2
- Place the centroid of the filter at the first pixel of the output (red 3x3 square)
- Paste the multiplied elements of the filter into the output positions (ignore the padded region).

Transpose Convolution



filter $f \times f = 3 \times 3$ padding $p = 1$ stride $s = 2$

Andrew Ng

Steps of the transpose convolution (previous slide)

Step 1:

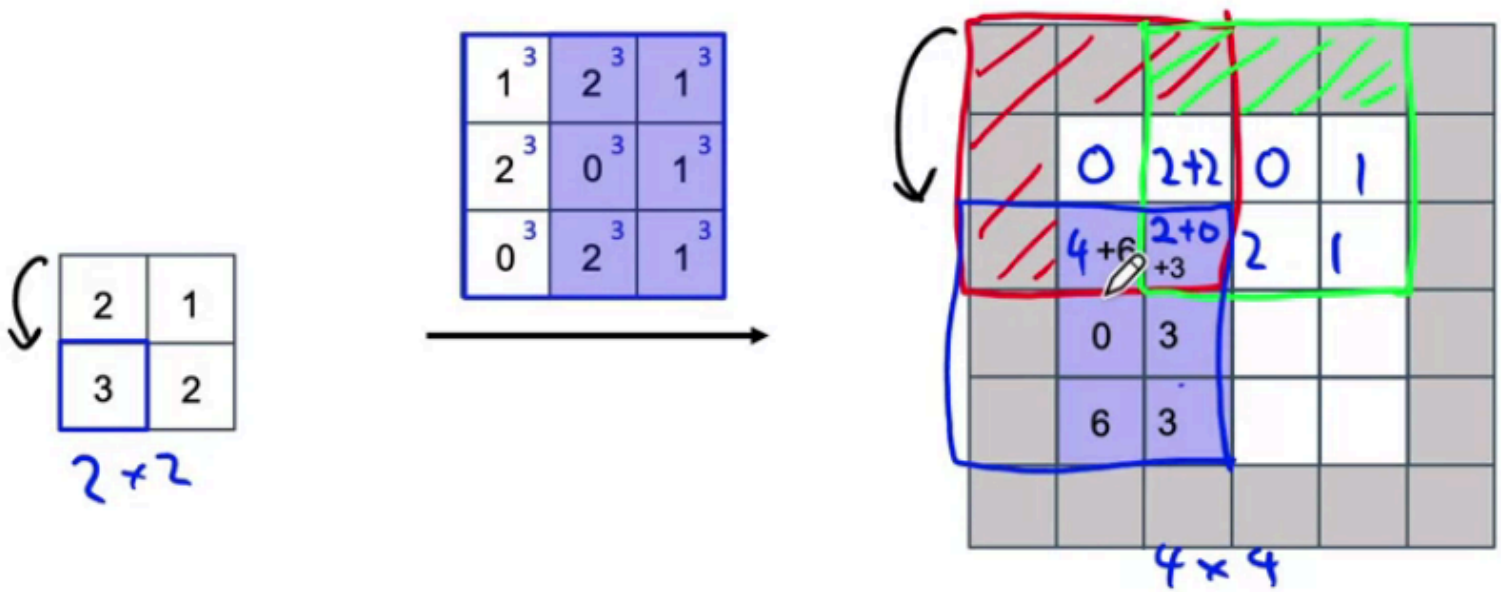
- Take the next element in the input (1 in the green square)
- Multiply the filter coefficients by 1

Step 2:

- Place the filter's center to 2 units to the right of previous position (since stride=2). Ignore the padded region.
- Copy the scaled filter coefficients in the green shaded region to the green shaded region in the output. If there's already a number in a position of the output add the current value to the previous one.

- Repeat similar steps for the blue element (3) in the input.

Transpose Convolution

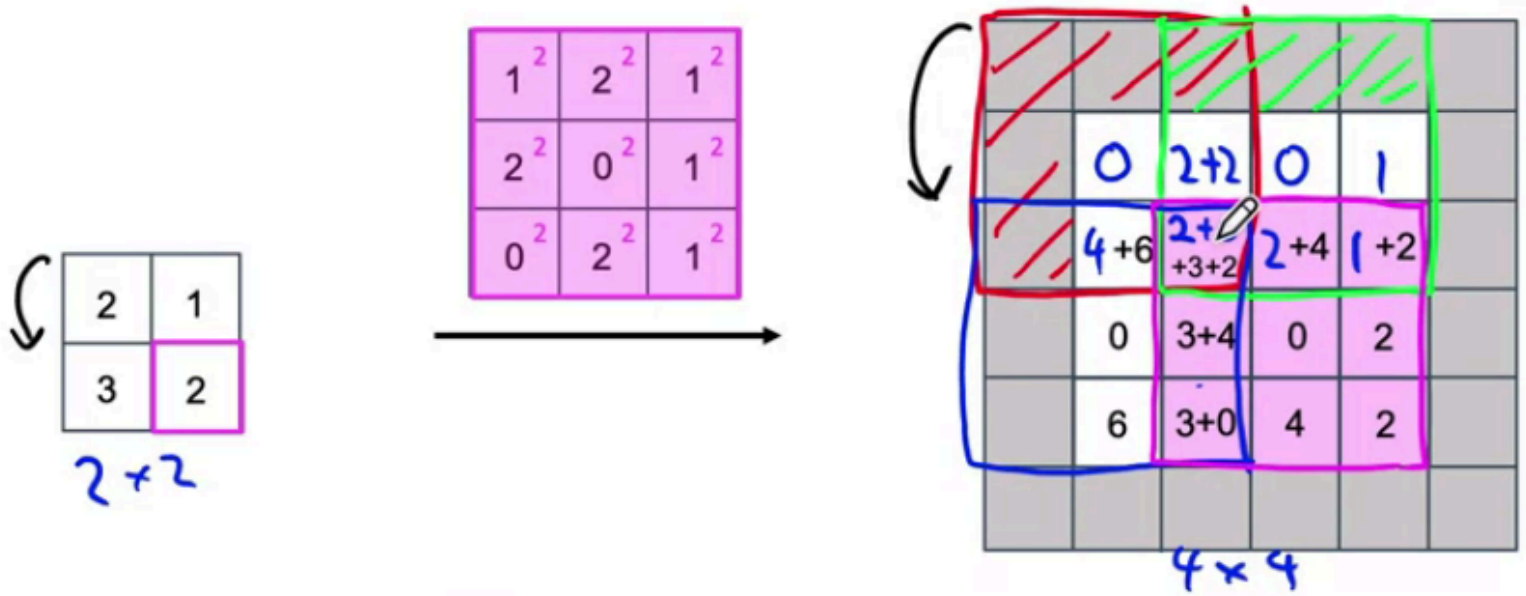


filter $f \times f = 3 \times 3$ padding $p = 1$ stride $s = 2$

Andrew Ng

• Repeat similar steps for the pink element (2) in the input.

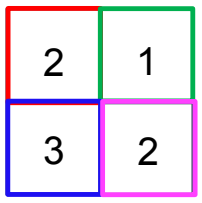
Transpose Convolution



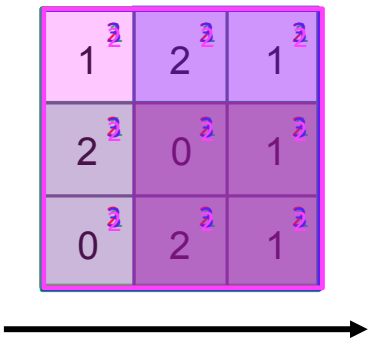
filter $f \times f = 3 \times 3$ padding $p = 1$ stride $s = 2$

Andrew Ng

Transpose Convolution

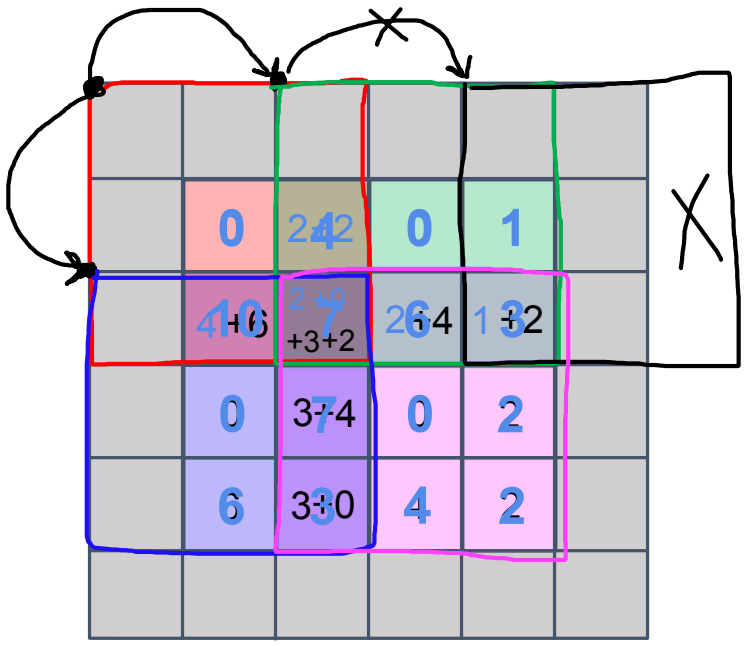


2 x 2



weight filter

add results of all computations



4 x 4

filter f x f = 3 x 3 padding p = 1 stride s = 2

Andrew Ng

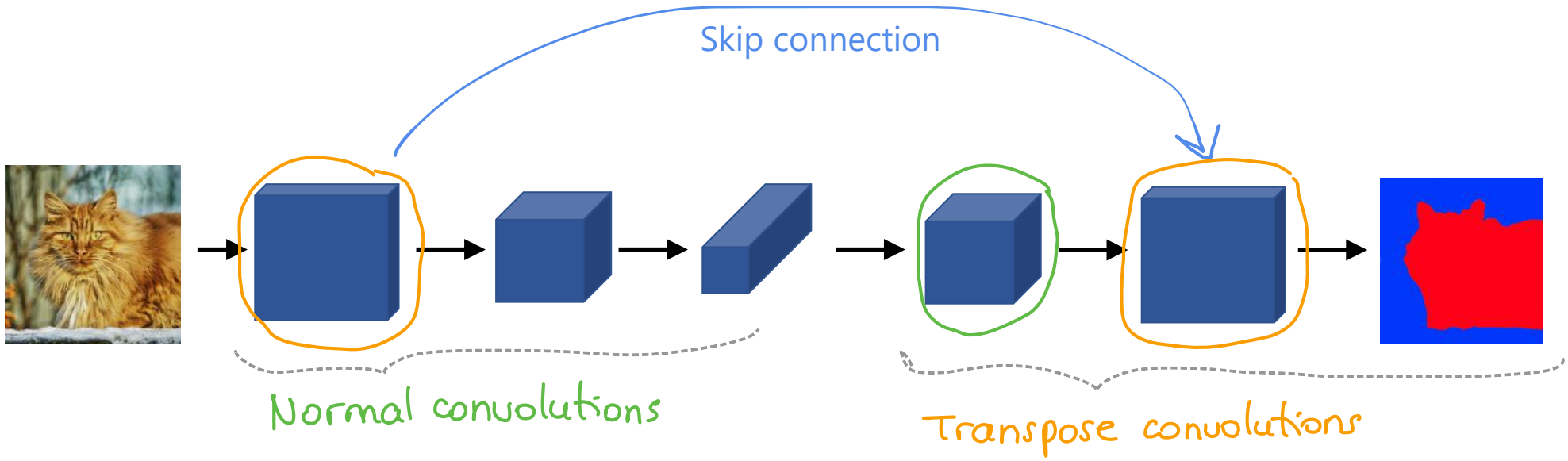
deeplearning.ai

Convolutional Neural Networks

U-Net Architecture intuition

Andrew Ng

Deep Learning for Semantic Segmentation



Skip connection: The earlier block of activations is copied directly to the later block. Why do we do this?

Andrew Ng

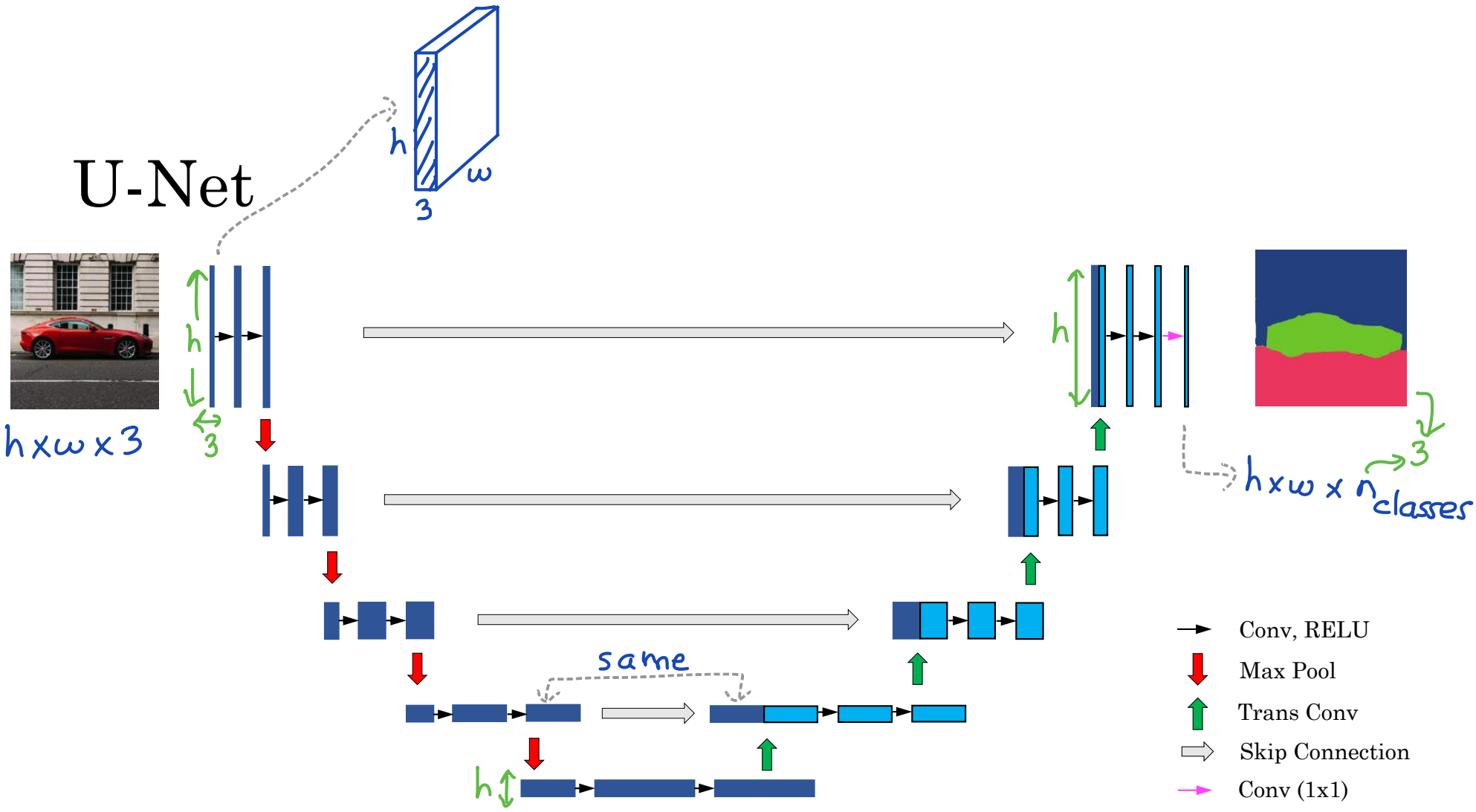
To decide which pixel belongs to a cat two types of information are useful:

1) Low resolution but high-level spatial contextual information from previous layer (circled with green). But detailed spatial information is missing since the height and width is lower.
2) High-resolution but low-level texture-like information (for every pixel how much hairy stuff is there?) (Comes from the early layer circled with yellow)

Convolutional Neural Networks

U-Net Architecture

[Ronneberger et al., 2015, U-Net: Convolutional Networks for Biomedical Image Segmentation]



[Ronneberger et al., 2015, U-Net: Convolutional Networks for Biomedical Image Segmentation]

- The final output of U-Net has dimensions $h \times w \times n_{classes}$. The $n_{classes}$ numbers tells for each pixel how likely is that pixel to come from each of the different classes.
- If you take argmax over these n classes, you can classify each of the pixels into one of the classes, and you can visualize it like the segmentation map shown on the right.